学号：202310310083

上海海事大学

航运应用软件开发课程设计



论文题目：航运物流公司管理系统

姓　　名：倪嘉宏

学　　院　信息工程学院

专　　业：计算机科学与技术

班　　级：计算机 231 卓

指导教师：金世双

完成时间：2026　年　5　月

论文独创性声明

本论文是我个人在导师指导下进行的研究工作及取得的研究成果。论文中除了特别加以标注和致谢的地方外，不包含其他人或者其他机构已经发表或撰写过的研究成果。其他同志对本研究的启发和所做的贡献均已在论文中作了明确的声明并表示了感谢。

作者签名：倪嘉宏 日期：2026/6/1

论文版权使用授权书

本论文作者完全了解学校有关保留、使用论文的规定，即：学校有权保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权上海海事大学可以将本论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本论文。

本学位论文属于（请在以下相应方框内打“√”）：

☑ 保密，在 2 年解密后适用本使用授权书。

☐ 不保密。

作者签名：倪嘉宏 导师签名：金世双 日期：2026/6/1

摘要

随着现代航运物流行业信息化与数字化水平的不断提高，传统依靠人工记录与线下沟通的物流管理方式已经难以满足企业对于运输效率、业务协同以及数据可追溯性的需求。为提高航运物流企业的运营效率与管理水平，本文设计并实现了一套基于 Vue 3 与 Django 的航运物流公司管理系统。系统采用前后端分离架构，前端基于 Vue 3 构建响应式单页应用，实现用户交互、业务展示以及数据可视化功能；后端基于 Django 框架提供 RESTful API 服务，负责业务逻辑处理、权限控制与数据管理；数据库采用 MySQL，用于存储订单、任务、货物、港口、客户以及用户等核心业务数据。

系统主要包含订单管理、客户管理、港口管理、货物管理、任务指派、运输跟踪、故障上报、通知中心、用户权限管理以及统计分析等功能模块。根据实际业务需求，系统划分为操作员、指派员、运输员和管理员四类角色，不同角色拥有不同的业务权限与操作范围，从而实现系统功能的规范化管理。在数据展示方面，系统结合 ECharts 实现订单趋势分析、任务状态统计以及货物数据分析等可视化功能，帮助管理人员快速掌握企业运营情况，提高决策效率。

在系统开发过程中，重点解决了前后端接口联调、权限控制、数据分页查询以及图表动态渲染等问题，并通过表单校验、异常处理与操作确认机制提高系统稳定性与用户体验。系统测试结果表明，该系统能够较好地满足航运物流企业在订单处理、运输调度、运输监控以及数据统计分析等方面的业务需求，具备良好的可扩展性、可维护性以及一定的实际应用价值。

关键词：航运物流；Vue；Django；管理系统；数据可视化

Abstract

With the continuous improvement of informatization and digitalization in the modern shipping logistics industry, traditional logistics management methods relying on manual recording and offline communication can no longer meet the growing demands for transportation efficiency, business collaboration, and data traceability. In order to improve the operational efficiency and management level of shipping logistics enterprises, this paper designs and implements a Shipping Logistics Company Management System based on Vue 3 and Django.

The system adopts a front-end and back-end separation architecture. The front-end is developed using Vue 3 to build a responsive single-page application, providing user interaction, business operation interfaces, and data visualization functions. The back-end is implemented with the Django framework and provides RESTful API services responsible for business logic processing, permission control, and data management. MySQL is used as the relational database to store core business data such as orders, tasks, cargo information, ports, customers, and user accounts.

The system mainly includes modules such as order management, customer management, port management, cargo management, task assignment, transportation tracking, fault reporting, notification management, user permission management, and statistical analysis. According to practical business requirements, the system defines four user roles: operator, dispatcher, transporter, and administrator. Different roles are granted different permissions and operation scopes, ensuring standardized and secure business management. In terms of data presentation, ECharts is integrated to implement visual analysis functions including order trend analysis, task status statistics, and cargo data analysis, enabling managers to quickly understand operational conditions and improve decision-making efficiency.

During the development process, key issues such as front-end and back-end API integration, permission control, data pagination, and dynamic chart rendering were addressed. In addition, form validation, exception handling, and operation confirmation mechanisms were introduced to enhance system stability and user experience. System testing results show that the platform can effectively meet the business requirements of shipping logistics enterprises in areas such as order processing, transportation scheduling, transportation monitoring, and statistical analysis. The system also demonstrates good scalability, maintainability, and practical application value.

Key words: Shipping Logistics; Vue; Django; Management System; Data Visualization; Front-end and Back-end Separation

目录

1、系统概述.....1
2、系统需求分析.....1
2.1 系统设计目标.....1
2.2 需求分析.....1
3、系统设计.....2
3.1 系统功能设计.....2
3.2 数据库设计.....2
3.2.1 逻辑结构设计.....2
3.2.2 概念结构设计.....3
4、系统实现.....4
4.1 登录与注册.....4
4.1.1 登录界面.....4
4.1.2 注册界面.....5
4.1.3 忘记密码界面.....6
4.2 操作员界面.....8
4.2.1 操作员主界面.....8
4.2.2 客户管理.....11
4.2.3 港口管理.....14
4.2.4 货物管理.....16
4.2.5 订单管理.....17
4.2.6 系统设置.....19
4.3 指派员界面.....20
4.3.1 指派员主界面.....20
4.3.2 订单指派.....22
4.3.3 任务管理.....23
4.3.4 通知中心.....25
4.3.5 系统设置.....27
4.4 运输员界面.....27
4.4.1 指派员主界面.....27
4.4.2 任务中心.....27
4.4.3 位置上报.....28
4.4.4 故障上报.....29
4.4.5 系统设置.....30
4.5 管理员.....30
4.5.1 管理员主界面.....30
4.5.2 用户管理.....32
4.5.3 系统设置.....33
5.系统测试.....33
5.1 测试环境.....33
5.2 测试结果.....33
5.2.1 用户登录与注册.....33
5.2.2 管理员功能.....35

5.2.3 操作员功能.....37
5.2.4 指派员功能.....38
5.2.5 运输员功能.....40
6.系统改进.....41
6.1 课程设计存在的问题及解决思路.....41
6.2 进一步改进方向.....42
6.2.1 增加地图以进行更丰富的可视化分析.....42
6.2.2 完善通知系统（如短信、邮件提醒）.....42
6.2.3 提升前端用户体验.....42
6.2.4 加入异常自动处理与风险预警.....42
7．小结.....42
参考文献.....43
致谢.....44

1、系统概述

本系统基于 Vue.js + Django 构建，是面向海运物流行业的一体化管理平台，旨在为货运企业提供从订单处理、运输跟踪到仓储管理与数据分析的全流程数字化解决方案[1]。系统采用前后端分离架构：前端基于 Vue 3 构建单页应用，负责实现交互式业务界面、可视化数据看板以及运输状态的实时展示；后端依托 Django 提供 API 服务，承担核心业务逻辑、权限控制、数据处理与持久化管理[3]。系统以 MySQL 作为关系型数据库，集中存储订单信息、航次与车辆数据、仓储记录及用户权限等核心业务数据。

平台主要涵盖订单管理、运单跟踪、库存与仓储管理、用户及角色权限管理、消息通知与统计报表等核心模块，并支持基于角色的权限分配与操作审计机制，能够有效保障系统的数据安全性与业务规范性。整体架构设计清晰合理，模块之间耦合度低，具备良好的可扩展性与可维护性。

在用户体验方面，系统界面简洁直观，操作流程清晰流畅，结合实时数据展示与可视化分析功能，能够显著提升物流业务管理效率，为用户带来高效、稳定且友好的使用体验。

本系统为二人协作项目，具体分工如表 1.1 所示

表 1.1

小组成员	学号	分工内容
倪嘉宏	202310310083	部分前端界面设计、数据库表设计、前后端联调及界面交互优化、后端数据处理、系统测试
冯方舟	202310310183	部分前端界面设计、报表展示、各用户主界面设计、系统测试

2、系统需求分析

2.1 系统设计目标

《航运物流公司管理系统》旨在为航运物流企业提供一个高效、可追踪、可视化的业务管理平台，实现从订单创建→调度指派→运输执行→完成交付→数据可视化管理的全流程信息化[2]。

系统目标包括[9]：

1、提升订单处理效率：通过系统化录入、查询与统计，提高工作效率。

2、规范调度流程：将运输系统分为四级，权责分明，各司其职，调度规范。

3、实时监控运输动态：支持实时监控运输人员位置，降低运输风险，可以做到对任何货物都能追根溯源。

4、保障运输安全与可追溯性：提供运输时故障、位置上报的界面，面对突发情况能够快速得到响应。

5、支持运营决策分析：提供可视化数据看板，辅助管理层快速掌握业务运行情况。

2.2 需求分析

系统主要功能需求包括：

订单管理（操作员）：订单、货物、用户数统计，维护港口、货物和客户信息，创建、修改、删除、查询订单，个人信息维护

调度指派（指派员）：指派任务数据统计，查看待指派订单，创建任务后指派给运输人员，任务在途监控，上报的故障处理，个人信息维护

运输执行（运输员）：运输报表统计，接收任务，更新运输状态，位置、异常上报，个人信息维护

系统管理（管理员）：报表导出，订单趋势统计，货物数量与价值统计，任务准确率统计，用户管理、个人信息维护

3、系统设计

3.1 系统功能设计

系统功能结构图如图 3.1 所示



图 3.1 系统功能结构图

3.2 数据库设计

3.2.1 逻辑结构设计

根据系统的核心业务对象以及前期的概念结构设计，本系统的主要关系模式如下：

1.权限表 permission

permission(perm_id, perm_name)

2.角色表 role

role(role_id, role_name)

3.用户表 user

user(user_id,username,password,role_id,phone,email,photo,status,create_time,login_status, last_visit)

4.角色-权限表 role_permission
role_permission (id,role_id,perm_id)
5.客户表 customer
customer(customer_id, customer_name,customer_phone,customer_address, customer_email)
6.港口表 port
port(port_id,port_code,port_name,port_name_en,country,city,latitude,longitude,contact_phone,contact_person,status,created_at,updated_at)
7.货物表 cargo
cargo(cargo_id,cargo_name,cargo_type,cargo_weight,cargo_volume,cargo_amount,inventory)
8.订单表 order
order(order_id,created_by,customer_id,task_id,origin_port,destination_port,order_status,total_amount,created_at,updated_at)
9.订单-货物表 order_cargo
order_cargo(id,cargo_id,order_id, quantity)
10.任务表 task
task(task_id,assigned_by,assigned_to,assigned_time,task_status,origin_port,destination_port, start_time)
11.轨迹跟踪表 track
track(track_id,task_id, latitude, longitude, recorded_at)
12.故障报告表 fault_report
fault_report(report_id,task_id,reported_by,description,reported_at,status)

3.2.2 概念结构设计

在系统需求分析的基础上，需要对系统中的核心业务对象及其相互关系进行抽象，从而形成数据库建设前的概念模型。概念结构设计的目标是识别系统中的关键实体、明确各实体所包含的属性，以及分析实体之间的联系类型（如一对一、一对多、多对多）。通过对这些信息进行统一建模，可以为后续的逻辑结构设计和数据库实现提供准确、清晰的结构基础。

为了更直观地展示系统的数据结构与业务关联，本小节将使用实体－联系模型对航运物流公司管理系统的概念结构进行描述。E-R 图能够有效地反映业务对象之间的层次关系和交互方式，是数据库设计过程中不可或缺的重要步骤[6]。
下面给出本系统的 E-R 图，用于展现主要实体及其之间的联系：

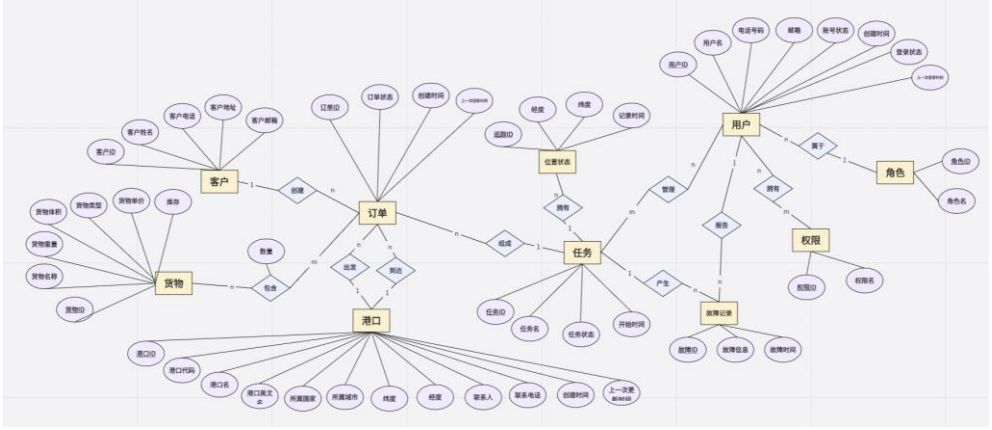


图 3.2 系统 E-R 图

4、系统实现

4.1 登录与注册

4.1.1 登录界面

本系统采用前后端分离架构实现用户认证功能。前端基于 Vue 3 Composition API 构建登录模块，通过双向绑定获取用户输入的用户名、密码，并通过封装后的 Axios 请求模块与后端进行交互[3]。

后端提供统一的 /api/login/ 登录接口，负责校验用户身份信息并返回认证结果及用户角色信息。前端在接收到响应后，根据返回的 success 状态判断登录是否成功。

若认证通过，系统将用户信息持久化存储至 localStorage，以便实现状态保持。同时根据用户角色进行权限路由分发，实现不同角色进入不同业务系统模块。

在安全性方面，请求使用 withCredentials: true 支持 Cookie 会话机制，结合后端可实现 Session 或 Token 认证方式，增强登录态管理能力。

此外，通过统一的错误捕获机制处理网络异常与认证失败情况，提高系统的稳定性与用户体验。

界面效果如下：



图 4.1 登录

响应式数据与基础依赖如下：

```
import { ref } from 'vue'
import { useRouter } from 'vue-router'
import { api } from "../utils/common.js"
const router = useRouter()
const username = ref('')
const password = ref('')
const remember = ref(false)
const msg = ref('')
const showPassword = ref(false)
```

登录核心逻辑：

```
const handleLogin = async () => {
  try {
    const res = await api.post('/api/login/', {
    username: username.value, password: password.value, remember: remember.value,
    }, {
      withCredentials: true
    })
    if (res.data.success) {
      msg.value = '登录成功'
      const role = res.data.user.role
      const user = res.data.user
      localStorage.setItem('user', JSON.stringify(user))
      if (role === '管理员') {
        router.push('/admin')
      } else if (role === '操作员') {
        router.push('/operator')
      } else if (role === '指派员') {
        router.push('/appointer')
      } else if (role === '运输员') {
        router.push('/transporter')
      } else {
        msg.value = '未知角色'
      }
    } else {
      msg.value = res.data.msg
    }
  } catch (e) {
    msg.value = '请求失败'
  }
}
```

4.1.2 注册界面

用户注册模块用于完成系统新用户的创建。系统前端采用响应式数据对象 form 统一管理用户输入信息，包括用户名、手机号、邮箱、角色及密码等字段，此后向后端接口发送 POST 请求完成注册操作。后端负责数据校验，并返回统一结构的响应结果。

在设计上，注册模块支持多角色选择机制，用户可在注册时明确自身身份，该角色信息将直接影响后续系统权限分配。

角色与表单数据定义：

```
const roles = ref(['管理员','操作员','指派员','运输员'])
const showPassword = ref(false)
const showPassword2 = ref(false)
const errors = ref([])
const form = ref({username: '',telephone: '',email: '',role: '',password: '',password2: ''})
```

注册核心逻辑

```
const handleRegister = async () => {
    errors.value = []
    try {
      const res = await api.post('http://127.0.0.1:8000/api/register/', form.value)
          alert(res.data.message)
          router.push('/')
      } catch (err) {
          if (err.response) {
              errors.value = err.response.data.errors
          } else {
              errors.value = ['服务器异常']
          }
        }
    }
```

图 4.2 为注册账号界面



图 4.2 注册账号

4.1.3 忘记密码界面

忘记密码模块用于帮助用户在遗忘登录密码时重新设置账户密码，是用户认证系统中的重要辅助功能。在功能设计上，系统要求用户输入用户名、新密码以及确认密码，并在前端首先完成“两次密码一致性”校验，以减少无效请求发送，提高用户体验。当前端校验通过后，系统通过 Axios 向后端 /api/retrieve_password/ 接口发送 POST 请求，由后端完成用户身份验证及密码更新操作。后端处理成功后返回统一响应结果，前端根据返回状态决定提示信息及页面跳转逻辑。

基础依赖与响应式变量：

```
import { ref } from 'vue'
import { useRouter } from 'vue-router'
import { api } from "../utils/common.js"
const router = useRouter()
const username = ref('')
const password = ref('')
const password2 = ref('')
const msg = ref('')
const show1 = ref(false)
const show2 = ref(false)
```

密码重置核心逻辑：

```
const handleSubmit = async () => {
  if (password.value !== password2.value) {
    msg.value = '两次密码不一致'
    return
  }
  try {
    const res = await api.post(
      'http://127.0.0.1:8000/api/retrieve_password/',
      {
        username: username.value,
        password: password.value,
        password2: password2.value
      }
    )
    if (res.data.success) {
      alert('修改成功')
      router.push('/')
    } else {
      msg.value = res.data.msg
    }
  } catch (e) {
    msg.value = '请求失败'
  }
```

图 4.3 为忘记密码界面

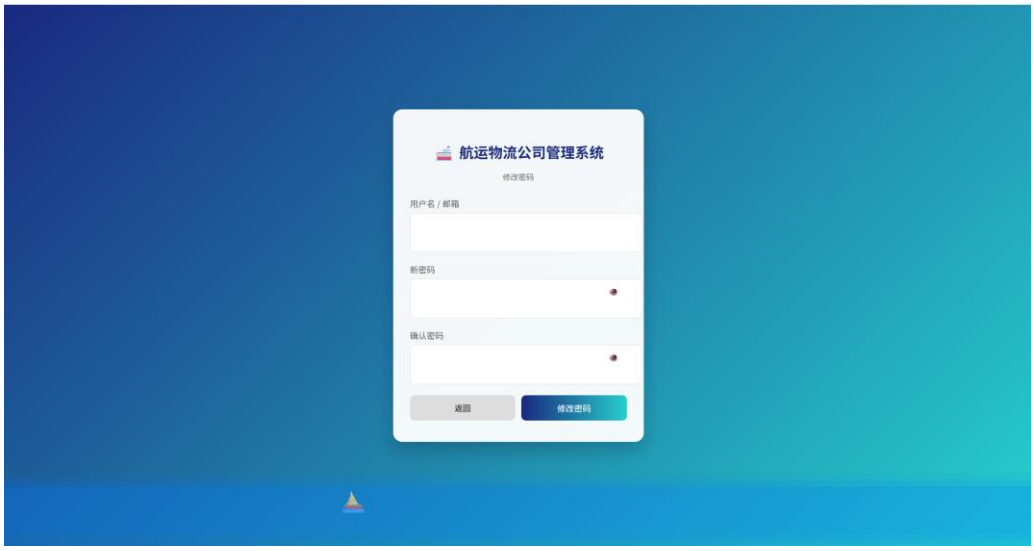


图 4.3 忘记密码

4.2 操作员界面

4.2.1 操作员主界面

该界面主要用于对系统中的业务数据进行汇总分析与图形化展示，帮助操作员快速了解系统运行状态与业务趋势，提高数据分析效率与管理决策能力。本系统基于 ECharts 实现前端数据可视化功能，通过 Axios 向后端接口请求统计数据，并采用饼图、折线图与柱状图等多种图表形式对数据进行动态展示。主要实现功能有：KPI 关键指标统计、订单状态分布分析、月度订单趋势分析、热门货物统计分析、时间区间筛选功能和图表自适应窗口缩放。同时，用户可通过选择开始时间与结束时间，对指定时间范围内的数据进行统计分析。前端将筛选参数通过 GET 请求传递给后端，由后端完成数据聚合与统计计算，再返回统一格式的数据结果。

图表与数据初始化：

```
import { ref, reactive, onMounted, nextTick } from 'vue'
import * as echarts from 'echarts'
import { api, fixAvatar } from "../utils/common.js"
const pieRef = ref(null)
const lineRef = ref(null)
const barRef = ref(null)
const user = reactive({name: '',role: '',avatar: ''})
const kpi = reactive({users: 0,orders: 0,cargo: 0,tasks: 0})
const startDate = ref('')
const endDate = ref('')
```

数据加载：

```
async function loadReport() {
  const params = {}
  if (startDate.value)
    params.start_date = startDate.value
  if (endDate.value)
    params.end_date = endDate.value
  const res = await api.get('/operator/report', {
    params
  })
  const data = res.data
  Object.assign(kpi, data.kpi)
  await nextTick()
  initCharts(data)
}
```

图表初始化实现

1、 订单状态饼图

```
pieChart = echarts.init(pieEl)
pieChart.setOption({
  tooltip: {
    trigger: 'item'
  },
  series: [{
    type: 'pie',
    radius: ['40%', '70%'],
    data: data.order_status.map(i => ({value: i.count,name: i.order_status}))
  }]
})
```

2、 月度订单趋势折线图

```
lineChart = echarts.init(lineEl)
lineChart.setOption({
  tooltip: {trigger: 'axis'},
  xAxis: {
    type: 'category', data: data.monthly_orders.map(i => i.month)
  },
  yAxis: {
    type: 'value'
  },
  series: [{
    type: 'line', smooth: true, data: data.monthly_orders.map(i => i.count)
  }]
})
```

3、 热门货物柱状图

```
barChart = echarts.init(barEl)
barChart.setOption({
    tooltip: {trigger: 'axis'},
    xAxis: {
        type: 'category', data: data.cargo_top.map(i => i.cargo__cargo_name)
    },
    yAxis: {
        type: 'value'
    },
    series: [{type: 'bar',data: data.cargo_top.map(i => i.total)}]
})
```

响应式窗口适配，当图表大小被改变，重新绘制图表：

```
window.addEventListener('resize', () => {
    pieChart?.resize()
    lineChart?.resize()
    barChart?.resize()
})
```

页面初始化：

```
onMounted(async () => {
    await loadReport()
    const userRes = await api.get('/api/user_info/')
    user.value = fixAvatar(userRes.data)
    Object.assign(user, userRes.data)
})
```

图 4.4 为操作员主界面

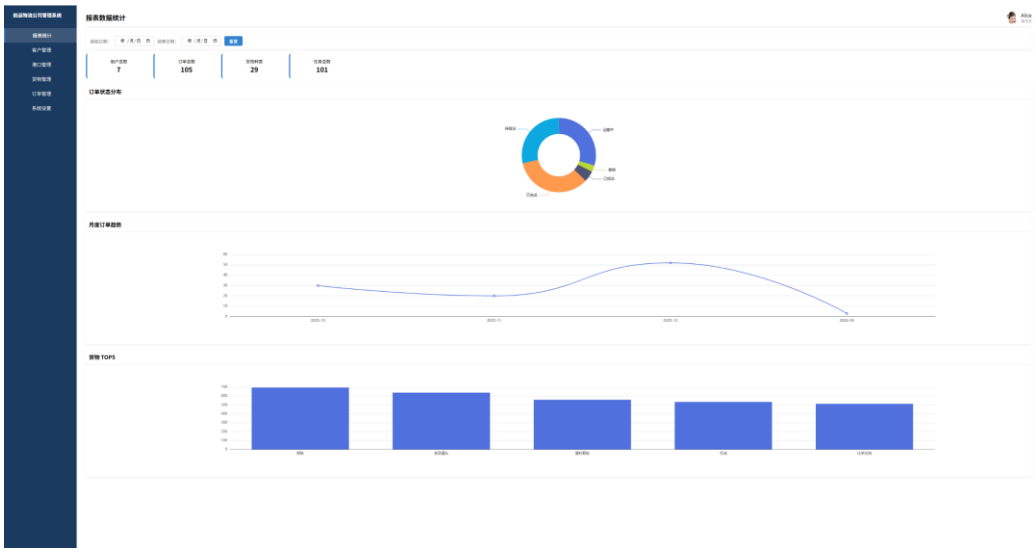


图 4.4 操作员主界面

4.2.2 客户管理

客户管理模块主要用于实现系统中客户信息的统一管理，包括客户数据查询、分页展示、信息编辑以及删除等功能。该模块能够帮助管理人员快速维护客户资料，提高物流业务的信息化管理效率。

系统采用 Vue 3 Composition API 实现前端页面逻辑，通过 Axios 与后端 API 接口进行数据交互。页面加载时，系统自动向后端请求客户数据，并结合分页机制实现大量客户数据的分批展示，减少页面一次性加载的数据量，提高系统运行效率。

在功能设计上，系统主要实现了：客户信息列表展示、客户关键字搜索、分页查询、客户信息编辑、客户信息删除和用户信息加载

其中，分页模块采用动态页码生成机制，仅显示当前页附近的页码范围，提高用户体验。删除功能采用二次确认机制，防止用户误操作导致数据丢失。

基础数据定义：

```
import { ref, reactive, onMounted } from 'vue'
import { useRouter } from 'vue-router'
import { api, fixAvatar } from "../utils/common.js"
const router = useRouter()
const list = ref([])
const page = ref(1)
const totalPages = ref(1)
const keyword = ref('')
const user = reactive({name: '',role: '',avatar: ''})
```

动态分页生成：

```
const pageRange = ref([])
const buildPageRange = (cur, total) => {
    const range = []
    const start = Math.max(1, cur - 2)
    const end = Math.min(total, cur + 2)
    for (let i = start; i <= end; i++) range.push(i)
    pageRange.value = range
}
```

客户数据加载：

```
const loadData = async (p = 1) => {                    //调用：loadData(p)
    const res = await api.get('/api/customers/', {
        params: {page: p,keyword: keyword.value}
    })
    const data = res.data.data
    list.value = data.customers
    page.value = data.pagination.current_page
    totalPages.value = data.pagination.total_pages
    buildPageRange(page.value, totalPages.value)
}
```

客户编辑跳转和删除功能

```
编辑跳转：
const goEdit = (id) => {
  router.push(`/customer/edit/${id}`)
}
删除：
const deleteCustomer = async (id) => {
  if (!confirm('确定删除？'))
    return
  await api.delete(`/api/delete-customers/${id}/`)
  loadData(page.value)
}
```

初始化：

```
onMounted(async () => {
  await loadData(1)
  const userRes = await api.get('/api/user_info/')
  user.value = fixAvatar(userRes.data)
  Object.assign(user, userRes.data)
})
```

图 4.5 为客户管理界面

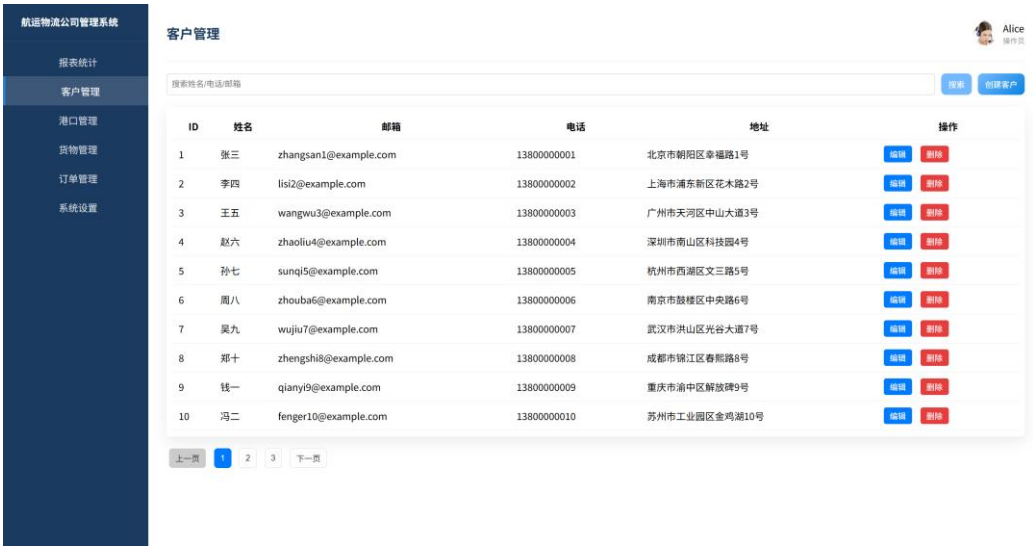


图 4.5 客户管理

客户编辑界面：

客户信息编辑模块主要用于对系统中的客户资料进行修改与维护，是客户管理功能的重要组成部分。系统通过获取指定客户的详细信息，并将数据回显至编辑表单，实现客户数据的更新与删除操作。系统通过路由参数获取客户 ID，并根据该 ID 请求对应客户详细信息，实现编辑页面的数据初始化，在修改完后调用 api 提交表单完成对后台数据库数据的更新。

基础数据定义：

```
import { reactive, onMounted } from 'vue'
import { useRoute, useRouter } from 'vue-router'
import { api, fixAvatar } from "../utils/common.js"
const router = useRouter()
const id = route.params.id
const user = reactive({name: '',role: '',avatar: ''})
const form = reactive({customer_name: '',customer_phone: '',customer_email: '',customer_address: ''})
```

客户详情加载

```
const loadCustomer = async () => {
  const res = await api.get(`/api/customers/${id}/`)
  Object.assign(form, res.data)
}
```

客户信息更新：

```
const submit = async () => {
  try {
    if (!form.customer_name ||!form.customer_phone ||!form.customer_address) {
      showMsg('必填项不能为空', 'error')
      return
    }
    if (form.customer_phone.length !== 11) {
      showMsg('手机号必须 11 位', 'error')
      return
    }
    await api.put(`/api/edit-customers/${id}/`,form)
    showMsg('更新成功')
    setTimeout(() => {router.push('/customer')}, 800)
  } catch (e) {
    showMsg('更新失败', 'error')
  }
}
```

表单重置功能：

```
const reset = () => { loadCustomer()}
```

初始化：

```
onMounted(async () => {
  await loadCustomer()
  const userRes = await api.get('/api/user_info/')
  user.value = fixAvatar(userRes.data)
  Object.assign(user, userRes.data)
})
```

图 4.6 为编辑客户界面

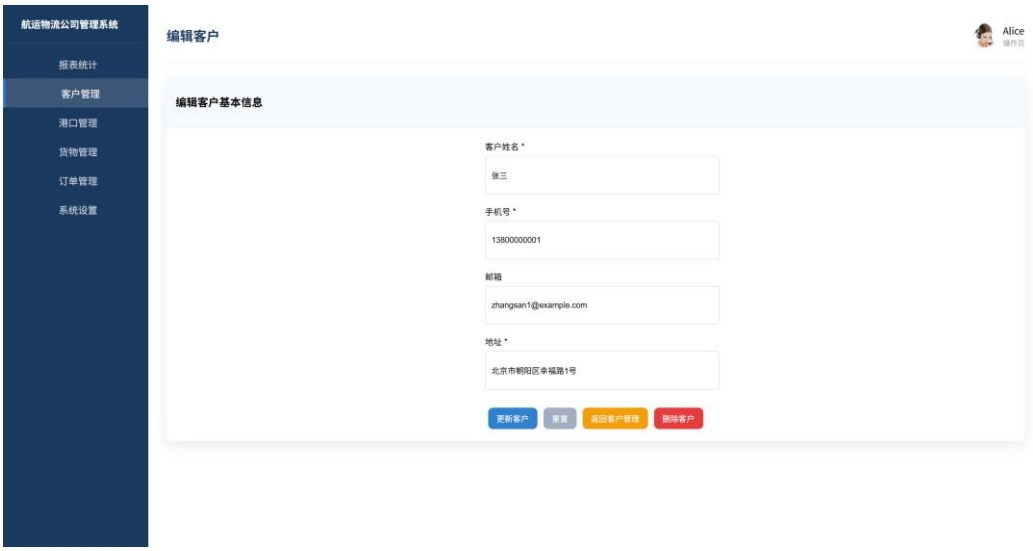


图 4.6 编辑客户

客户创建界面：

与客户编辑一致，客户填写表单后发送 POST 请求提交表单，后端根据表单信息进行合法性检验，并存储到数据库中。图 4.7 为创建客户界面。

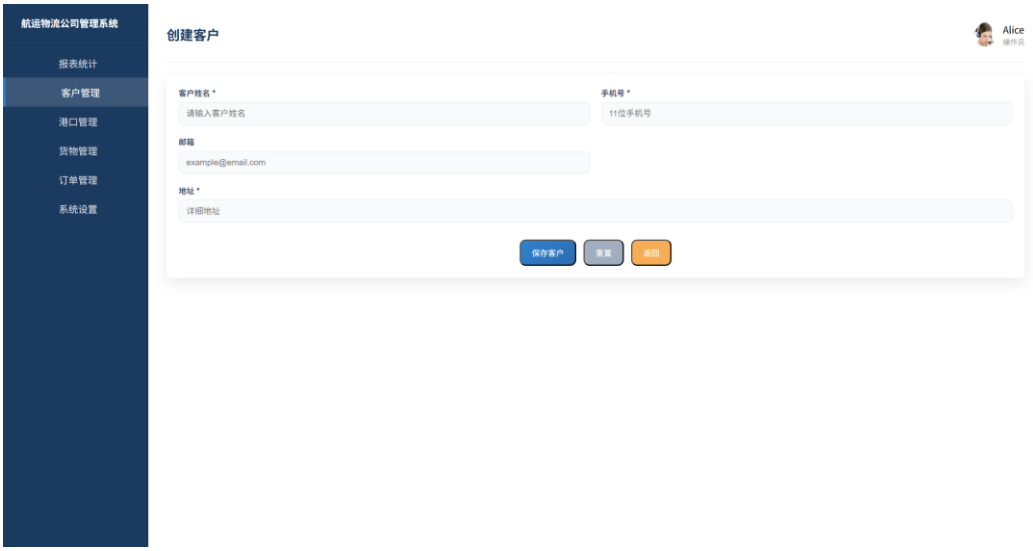


图 4.7 创建客户

4.2.3 港口管理

作为航运物流管理系统，港口管理在本系统中是十分重要的，因此操作员需要维护港口信息，港口管理设计思路与客户管理设计思路一致，图 4.7 是港口管理界面

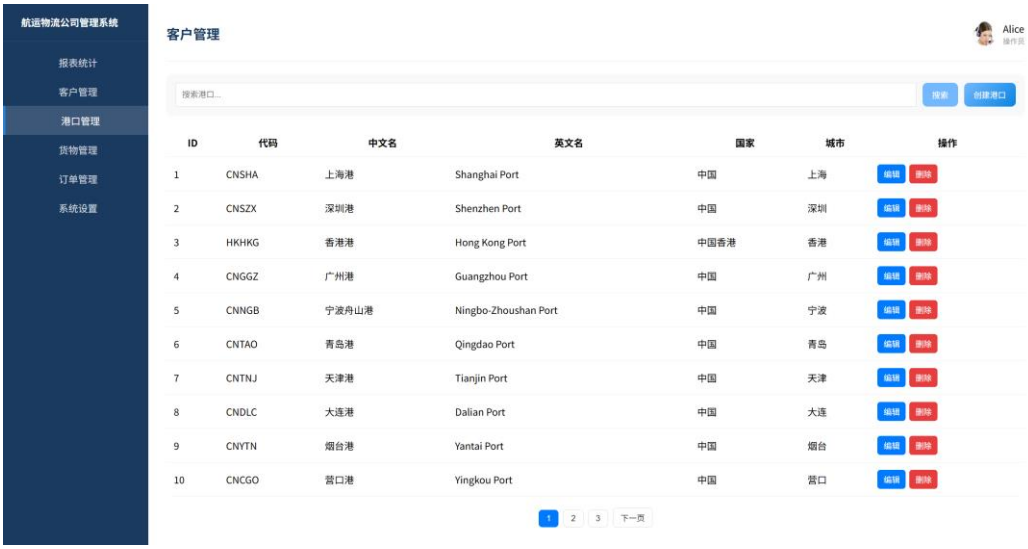


图 4.7 港口管理

图 4.8 为编辑港口界面

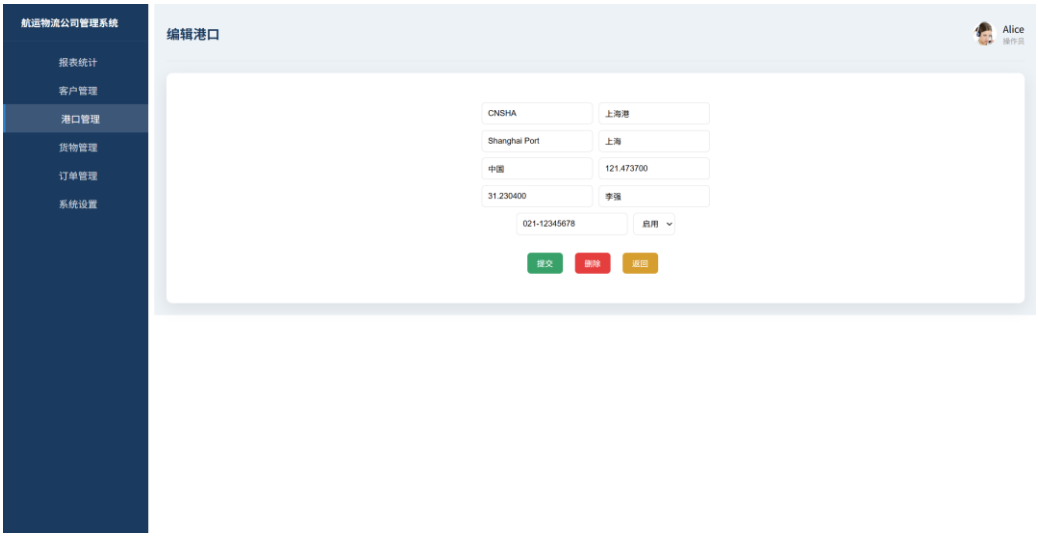


图 4.8 编辑港口

图 4.9 为创建港口界面

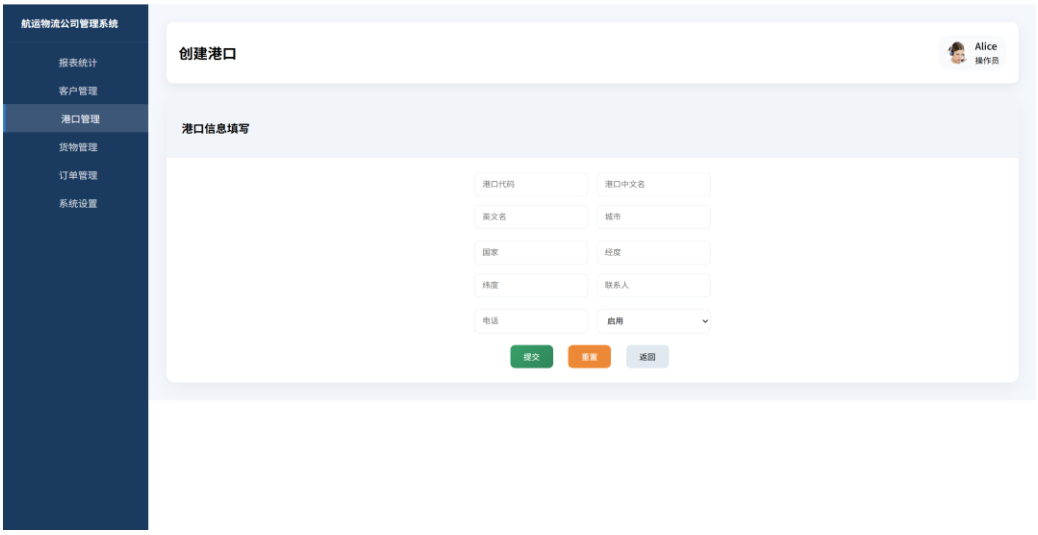


图 4.9 创建港口

4.2.4 货物管理

货物管理与客户、港口管理类似，不同点在于后端对重量、体积、单价、库存的合法性校验，前端响应式设计几乎保持一致。

图 4.10 为货物管理界面

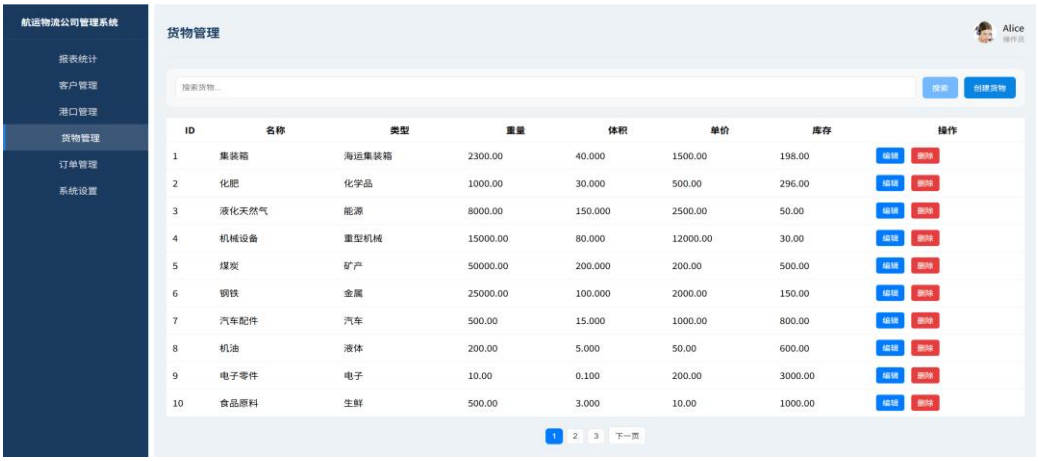


图 4.10 货物管理

图 4.11 为货物编辑界面

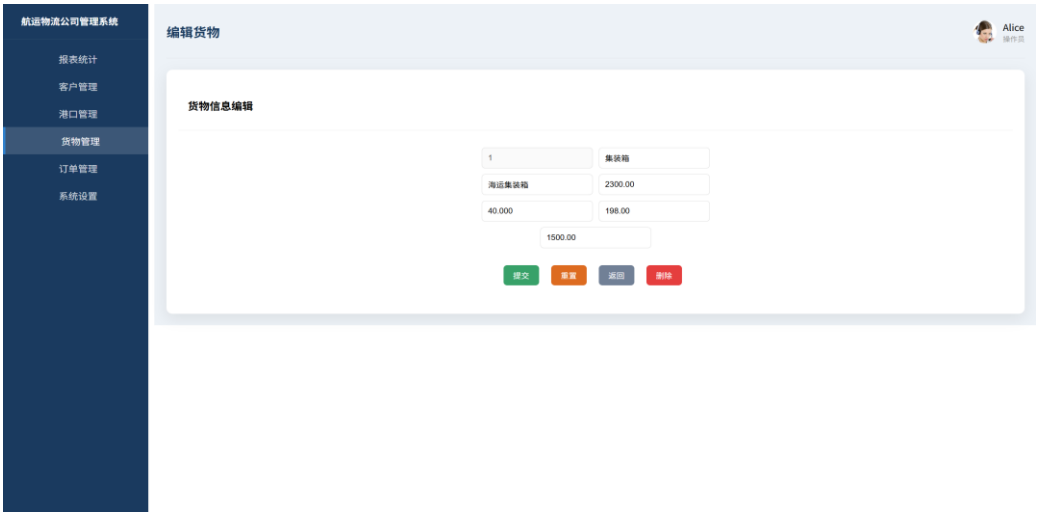


图 4.11 货物编辑

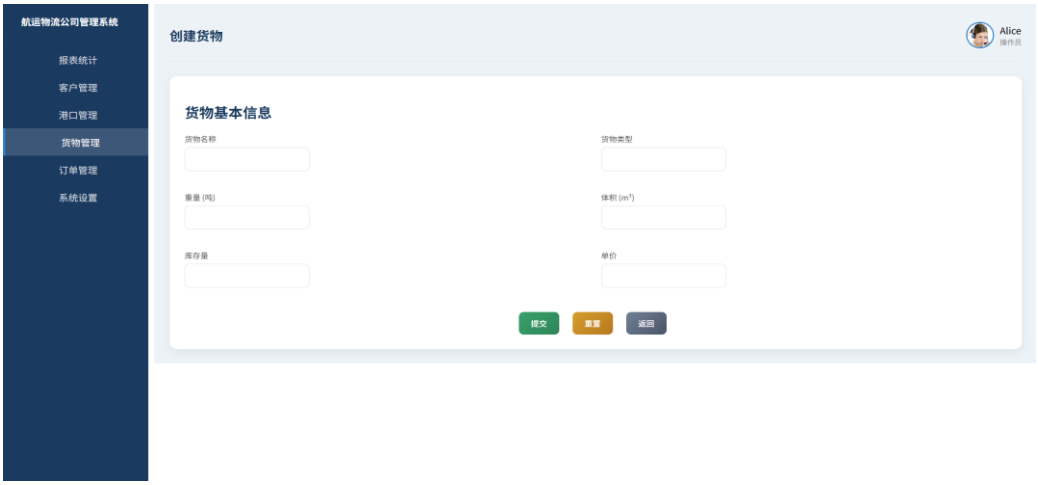


图 4.12 货物创建

4.2.5 订单管理

在订单展示界面与前面保持一致，调用 api 结合分页机制实现大量客户数据的分批展示,界面如图 4.13 所示

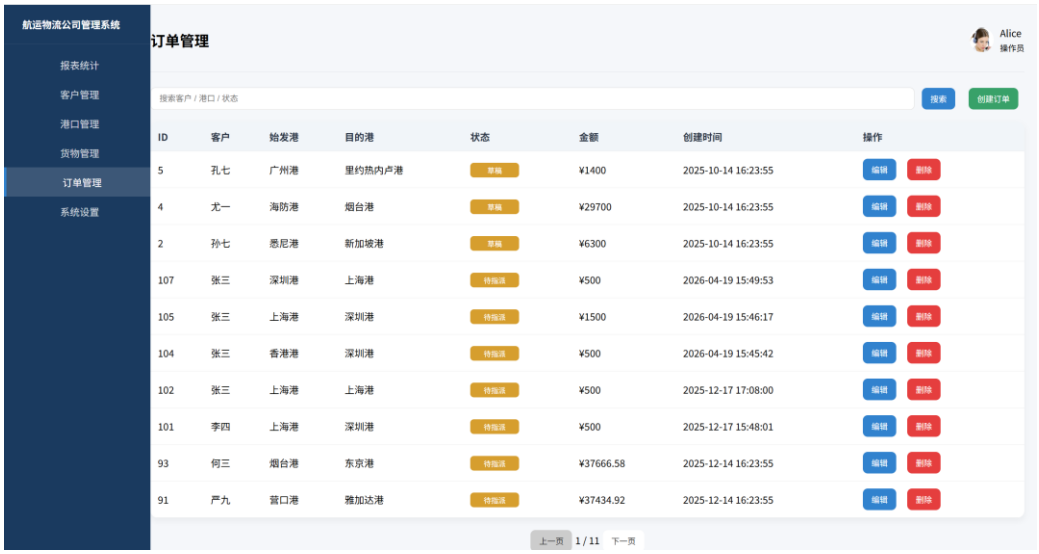


图 4.13 订单管理

在创建订单时，港口与货物搜索采用防抖机制，减少接口请求频率，提高系统性能与用户体验。核心代码如下：

搜索港口及选择

```
const onSearchPort = () => {              //搜索
  clearTimeout(portTimer)
  const val =(
      showOriginList.value
      ? originPortText.value
      : destPortText.value) || ''
    portTimer = setTimeout(async () => {
      const res = await api.get('/api/ports/', {params: {keyword: val}
    })
    portResults.value = res.data.list || []
  }, 200)
}
const selectPort = (which, p) => {              //选择
  if (which === 'origin') {
    originPortText.value = p.port_name    form.value.origin_port = p.port_id
    showOriginList.value = false
  } else {
    destPortText.value = p.port_name    form.value.destination_port = p.port_id
    showDestList.value = false
  }
}
```

```
const onSearchPort = () => {              //搜索
  clearTimeout(portTimer)
  const val =(
      showOriginList.value
      ? originPortText.value
      : destPortText.value) || ''
    portTimer = setTimeout(async () => {
      const res = await api.get('/api/ports/', {params: {keyword: val}
    })
    portResults.value = res.data.list || []
  }, 200)
}
const selectPort = (which, p) => {              //选择
  if (which === 'origin') {
    originPortText.value = p.port_name    form.value.origin_port = p.port_id
    showOriginList.value = false
  } else {
    destPortText.value = p.port_name    form.value.destination_port = p.port_id
    showDestList.value = false
  }
}
```

搜素货物：

```
const onSearchCargo = () => {                    //搜索
  clearTimeout(cargoTimer)
  cargoTimer = setTimeout(async () => {
    const res = await api.get('/api/cargos/', {
      params: {keyword: cargoText.value || ''}
    })
    cargoResults.value = res.data.list || []
    showCargoList.value = true
  }, 200)
}
```

添加货物：

```
const selectCargo = (c) => {
  const exist = cargoList.value.find(
    x => x.cargo_id === c.cargo_id
  )
  if (exist) {
    alert('已添加')
    return
  }
  const qty = prompt('输入数量', '1')
  if (!qty || isNaN(qty))
    return
  cargoList.value.push({
    cargo_id: c.cargo_id,
    cargo_name: c.cargo_name,
    quantity: Number(qty)
  })
  cargoText.value = ''
  showCargoList.value = false
}
```

提交表单与客户管理类似

图 4.13 是创建订单界面

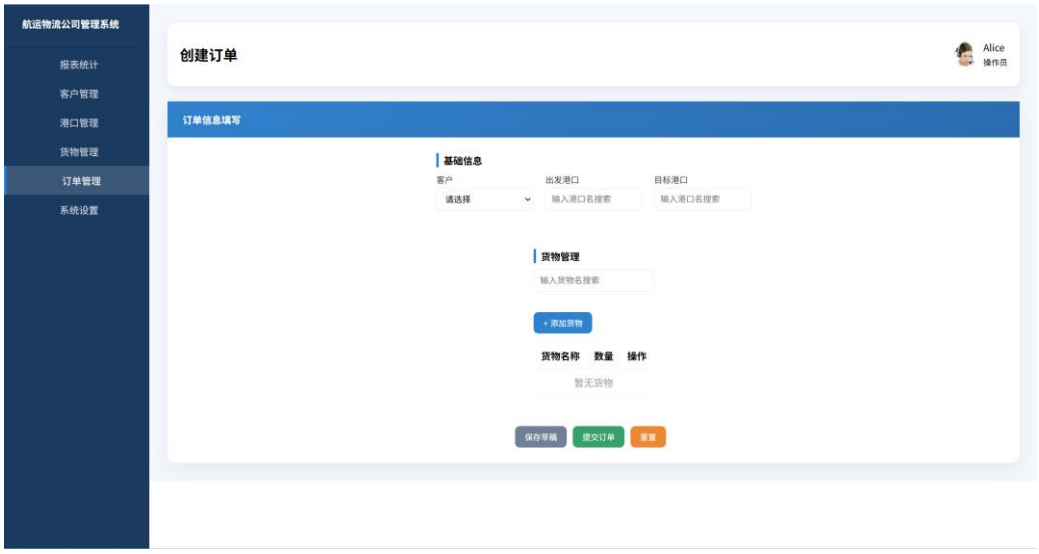


图 4.13 创建订单

4.2.6 系统设置

系统设置模块主要用于实现用户个人信息的查看与修改，包括用户基本资料更新及头像上传等功能，是系统中用于维护用户账户信息的重要组成部分。

同样通过 api 与后端接口进行数据交互，支持用户信息的获取与更新操作。系统在页面加载时自动请求用户个人信息，并将数据回显至界面.

在功能设计上，该模块主要实现以下内容：用户个人信息获取、用户资料修改、用户头像上传与预览、图片本地预览功能和用户退出登录功能。

在头像上传方面，系统采用 FormData 进行文件传输，实现二进制文件与文本数据的混合提交。同时在前端通过 URL.createObjectURL() 实现图片本地预览，提升用户体验。退出登录功能通过清除本地存储的用户信息实现，并跳转至登录页面，从而完成会话结束流程。

基础数据与接口配置和用户信息加载与之前保持一致。

图片选择与本地预览逻辑如下：

```
const triggerFile = () => {                    //选择
  fileInput.value.click()
}
const onFileChange = (e) => {                  //预览
  file.value = e.target.files[0]
  preview.value = URL.createObjectURL(file.value)
}
```

用户更新调用依旧是调用后端 api 发送表单实现。

用户退出登录逻辑如下：

```
const logout = () => {
  localStorage.removeItem('user')
  router.push('/')
}
```

图 4.14 是系统设置界面

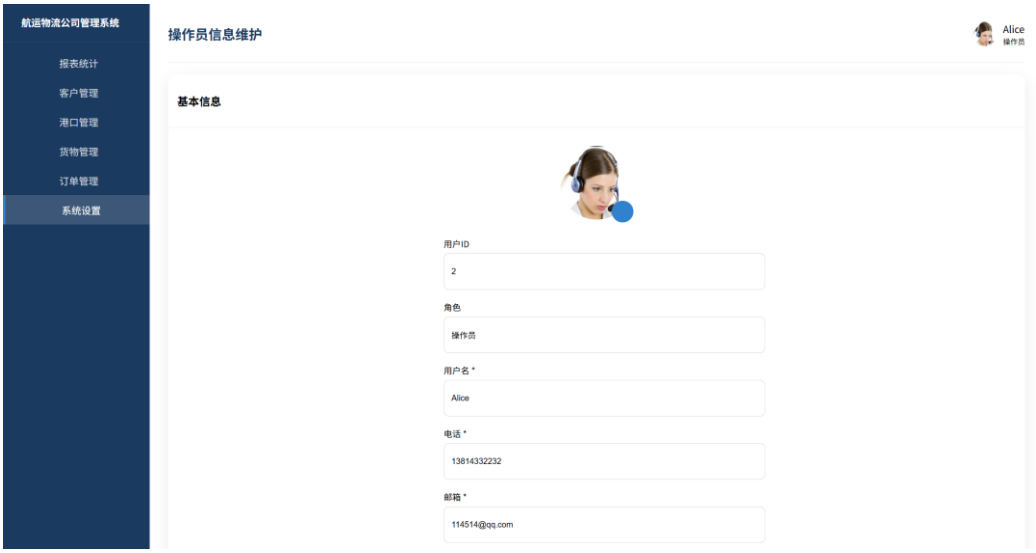


图 4.14 系统设置

4.3 指派员界面

4.3.1 指派员主界面

指派员主界面主要用于展示任务分配相关的核心业务数据，包括任务统计指标、任务状态分布以及最近任务列表，是指派员角色的核心工作台页面。

系统采用 ECharts 图表库实现数据可视化展示，通过 api 与后端接口进行数据交互，动态获取指派员工作台所需的统计信息与任务数据。

在功能设计上，该模块主要实现以下内容：指派员 KPI 数据展示、任务状态饼图可视化、最近任务列表展示、用户信息加载、任务状态动态样式渲染。系统在页面加载时同时请求两类数据：指派员仪表盘数据、当前登录用户信息。 其中任务状态采用不同样式标签进行区分，提升信息识别效率。ECharts 用于渲染任务状态分布饼图，使任务结构更加直观清晰。

任务状态饼图实现核心代码如下：

```
const initChart = () => {
  const chart = echarts.init(
    document.getElementById('statusChart')
  )
  chart.setOption({
    tooltip: {
      trigger: 'item'
    },
    legend: {
      top: '5%'
    },
    series: [
      {
        type: 'pie', radius: ['45%', '70%'],
        itemStyle: {
          borderRadius: 8, borderColor: '#fff', borderWidth: 2
        },
        data: pieData.value
      }
    ]
  })
}
```

图 4.15 为指派员主界面

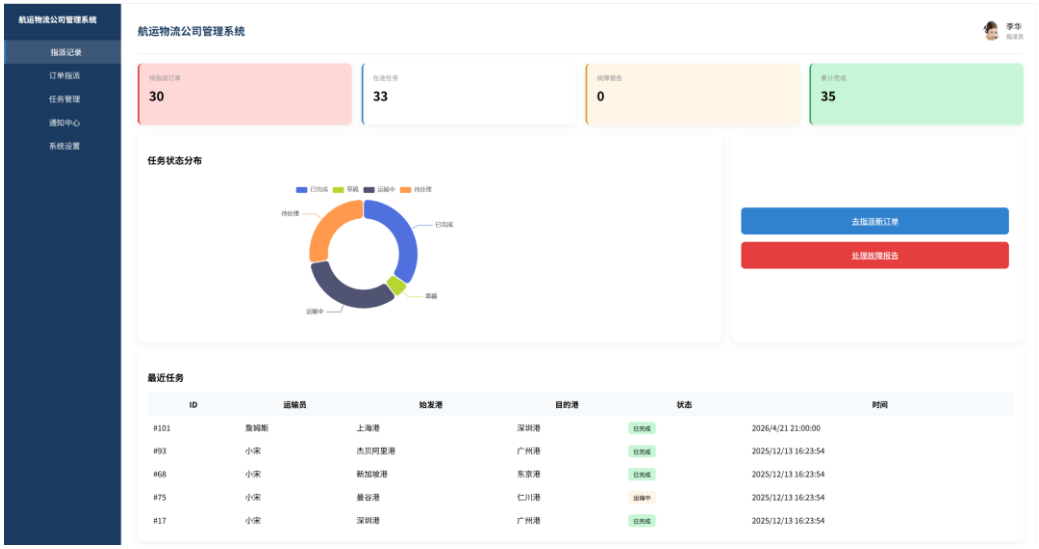


图 4.15 指派员主界面

4.3.2 订单指派

与客户管理类似，从后端返回订单数据，并结合分页机制实现订单数据的分批展示，减少页面一次性加载的数据量，提高系统运行效率。

图 4.16 为订单列表界面

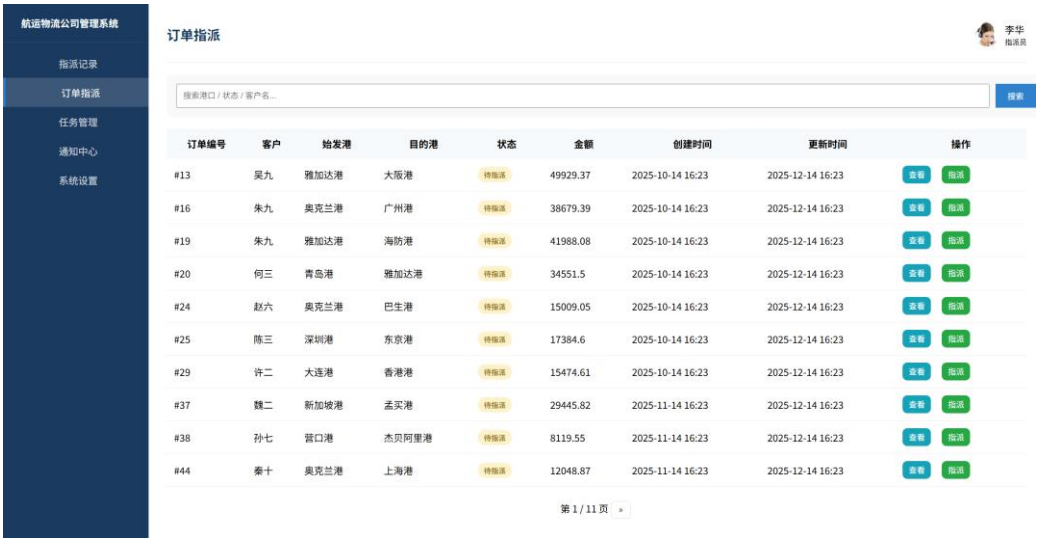


图 4.16 订单列表

点击右侧查看按钮就会看到该订单的详情，前端通过 api 调用后端接口，后端将从数据库中获取的数据加工后返回前端，前端接收到数据后回显在界面上。

图 4.17 为查看订单界面



图 4.17 查看订单

点击图 4.16 的指派按钮会转到对应订单的指派界面，可以选择任务对应的任务进行指派。

提交指派的逻辑如下，实现上，将用户选择将其分配的任务编号追加在表单后边传递给后端进行数据库交互。

```
const submit = async () => {
  if (!selectedTask.value) {
    msg.value = '请选择任务'
    success.value = false
    return
  }
  const id = route.params.id
  const form = new FormData()
  form.append('task_id', selectedTask.value)
  const res = await api.post(
    `/api/assign_order/${id}/submit/`,
     form)
  msg.value = res.data.message
  success.value = res.data.success
  if (res.data.success) {
    setTimeout(() => router.push('/assign_orders'), 1000)
  }
}
```

图 4.18 为订单指派界面

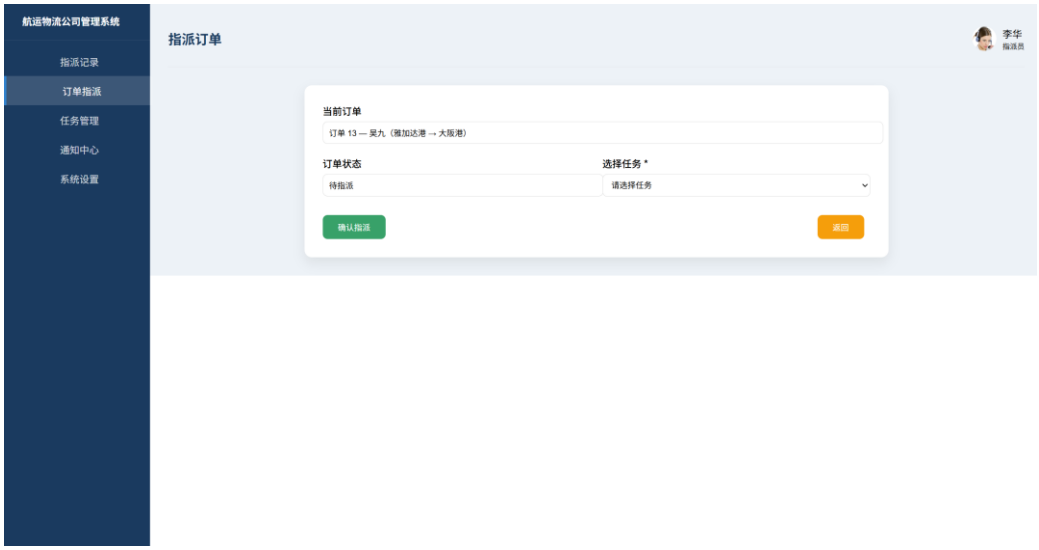


图 4.18 订单指派

4.3.3 任务管理

为了将不同订单集成到特定的任务，指派员的任务管理实现了以下几个功能：创建任务，查看任务详情与轨迹、编辑任务，任务展示。任务列表展示部分逻辑与客户展示逻辑一致。图 4.19 为任务列表展示界面。

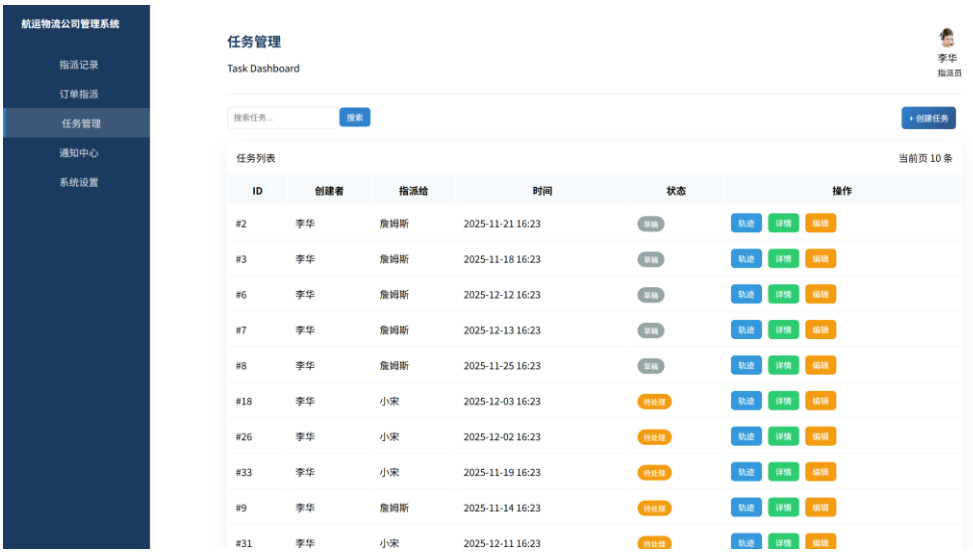


图 4.19 任务列表展示

图 4.20 是创建任务界面



图 4.20 创建任务



图 4.21 任务轨迹查看



图 4.22 任务详情

点击上图中每个订单后的“详情“按钮，就可跳转到图 4.17 的订单详情界面，而不用用户去订单管理搜索，增加用户体验。



图 4.23 任务编辑

4.3.4 通知中心

在航运过程中，难免会遇到突发情况，为了让运输员及时的查询指派的任务是否有问题，通知中心汲取了所有指派的任务的故障报告，操作员可以查看每条故障的详情信息以及处理过程中可以点击“处理”“完成”按钮修改状态[11]。

故障列表展示实现逻辑与客户管理保持一致，图 4.24 为故障上报列表展示界面

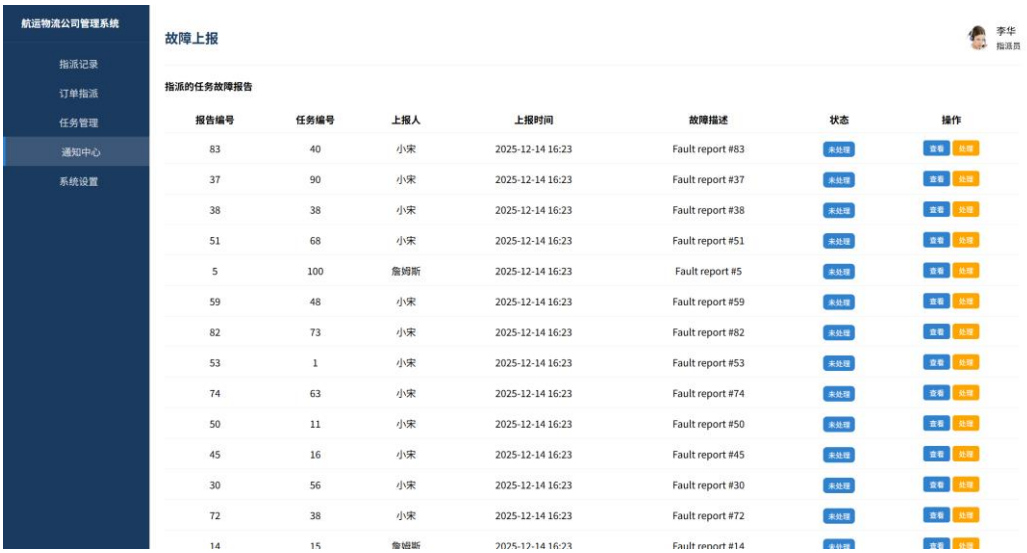


图 4.24 故障列表

点击上图的查看按钮就可以查看故障详情

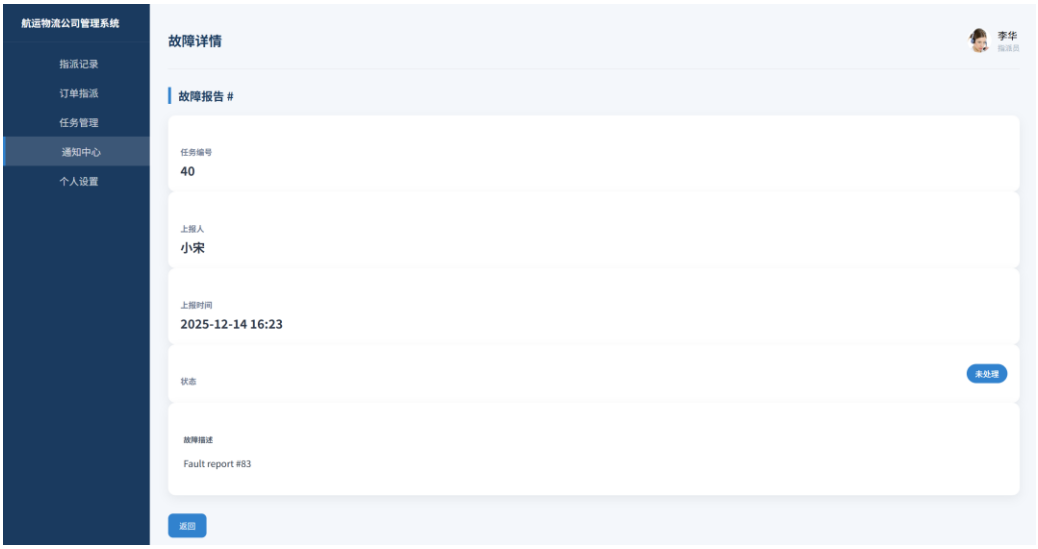


图 4.25 故障详情

点击“处理”按钮会将其状态改为处理中，“完成”会将状态改为已完成。



图 4.26 故障列表（续）

4.3.5 系统设置

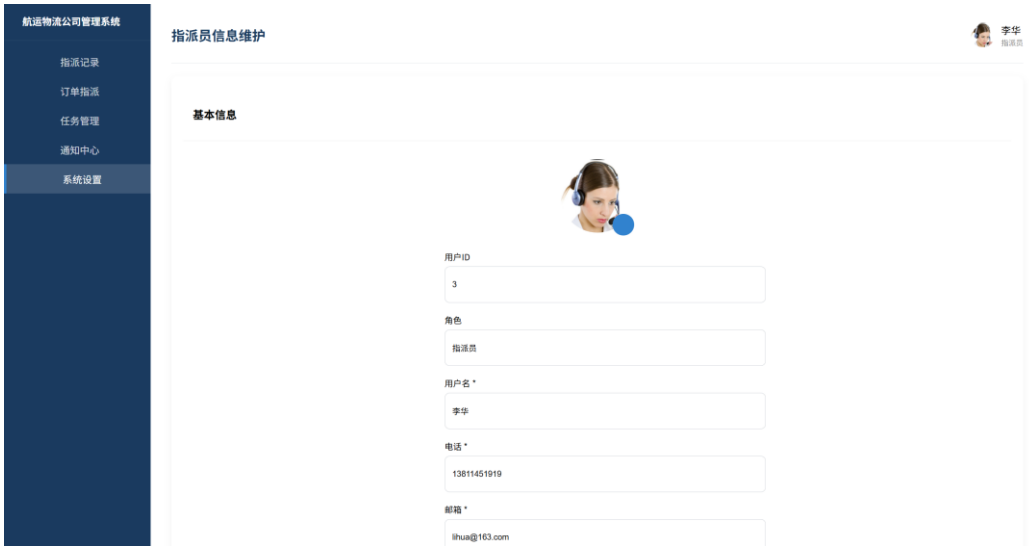


图 4.27 系统设置

4.4 运输员界面

4.4.1 指派员主界面

运输员主界面主要展示其累计接任务数、完成任务数、在途货物与累计上报故障数，以及在途任务信息，同样是调用后端 api 加载数据并展示。

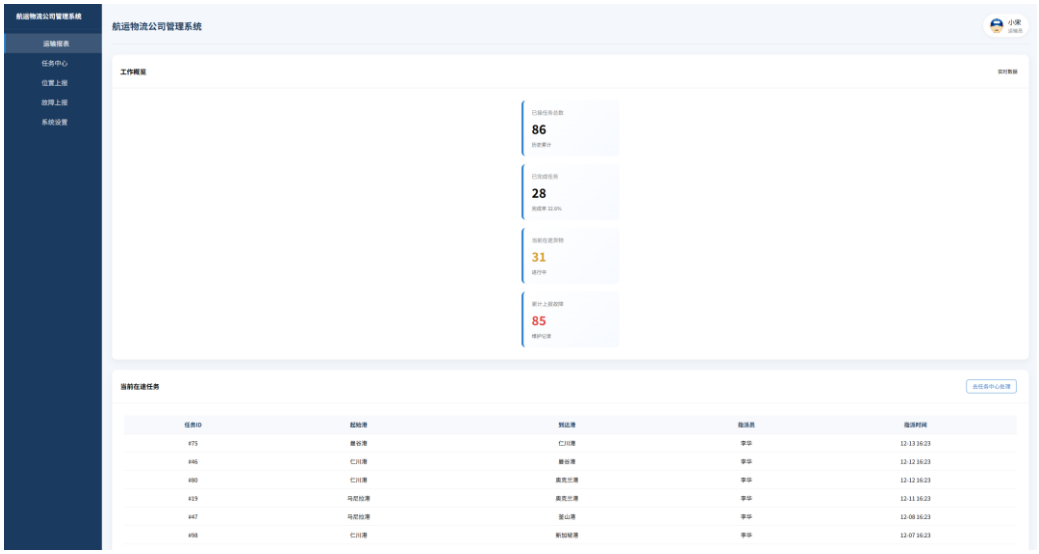


图 4.28 指派员主界面

4.4.2 任务中心

为了方便运输员查看自己的任务，并高效的查看任务信息，任务中心按行罗列了待处理、运输中、已完成的任务，展示过程与客户管理列表一致。

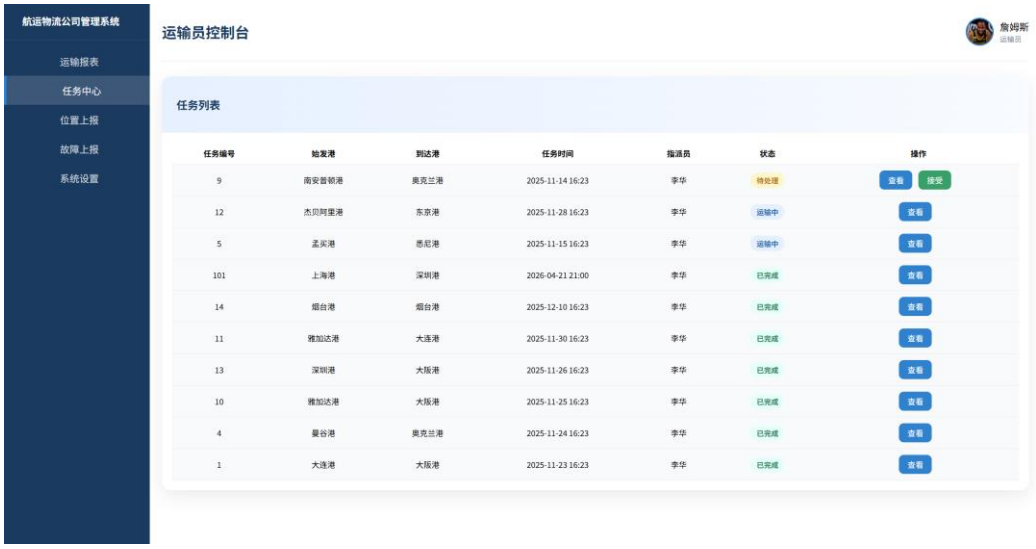


图 4.29 任务中心

点击操作栏中的查看按钮即可看到该任务下所有的订单与轨迹记录和任务详情信息。

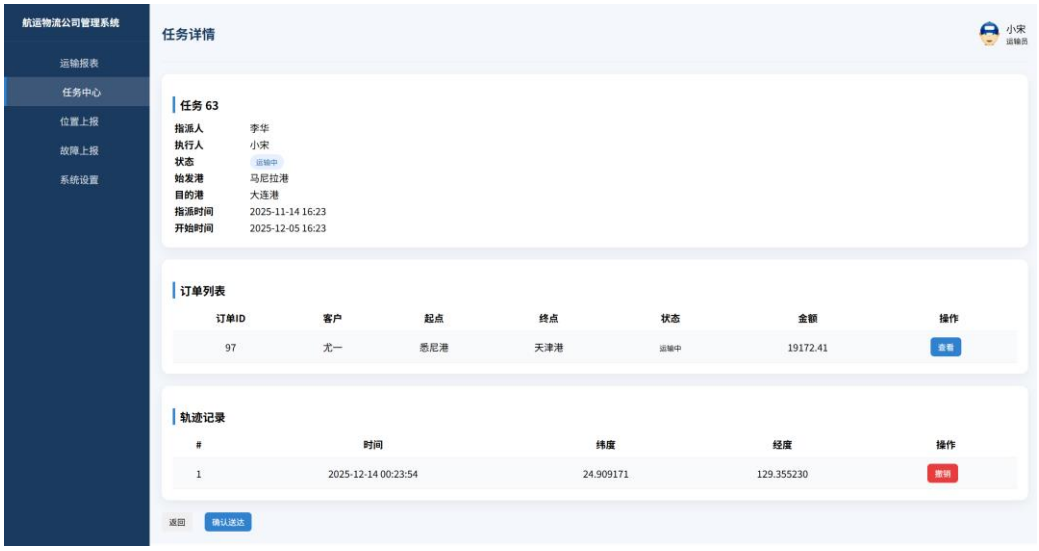


图 4.30 任务详情

点击订单的查看按钮即可查到图 4.17 的订单详情界面，点击轨迹操作中的撤销就会调用接口删除数据库中该条轨迹，对于运输中的货物，左下角有确认送达按钮，点击后该任务与订单状态都会被设置为已完成

对于图 4.29 中待处理的任务，操作栏有接受按钮，点击后即可将该任务及其包含的订单状态改为运输中。

4. 4. 3 位置上报

提供三个可填写表单，让用户选择任务，以及当前时间的的经纬度信息，当前用户和上报时间不可修改，自动获得当前登录的用户和系统时间点。填写完后用户发送 POST 请求就会调用 api 存入数据。

获取当前时间的代码如下：

```
const now = new Date().toLocaleString()
```

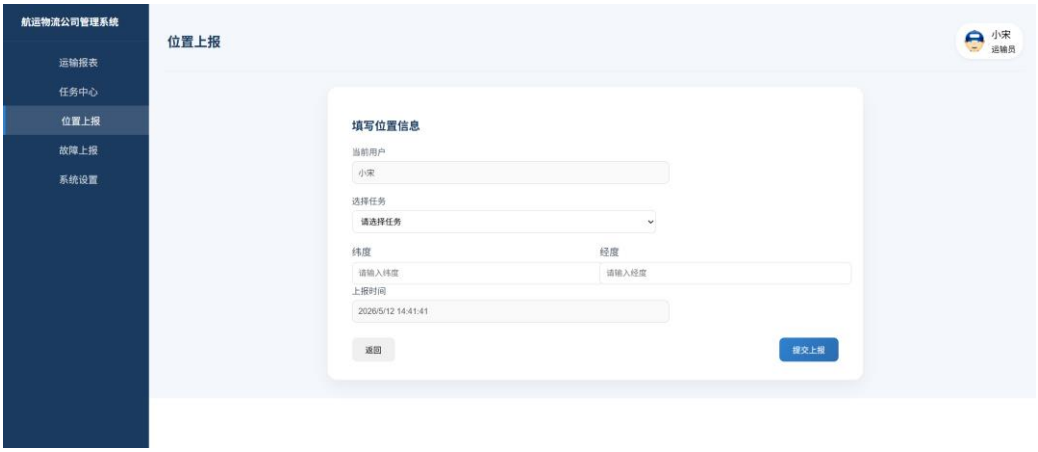


图 4.31 位置上报

4.4.4 故障上报

该列表展示了当前用户相关的所有故障信息。

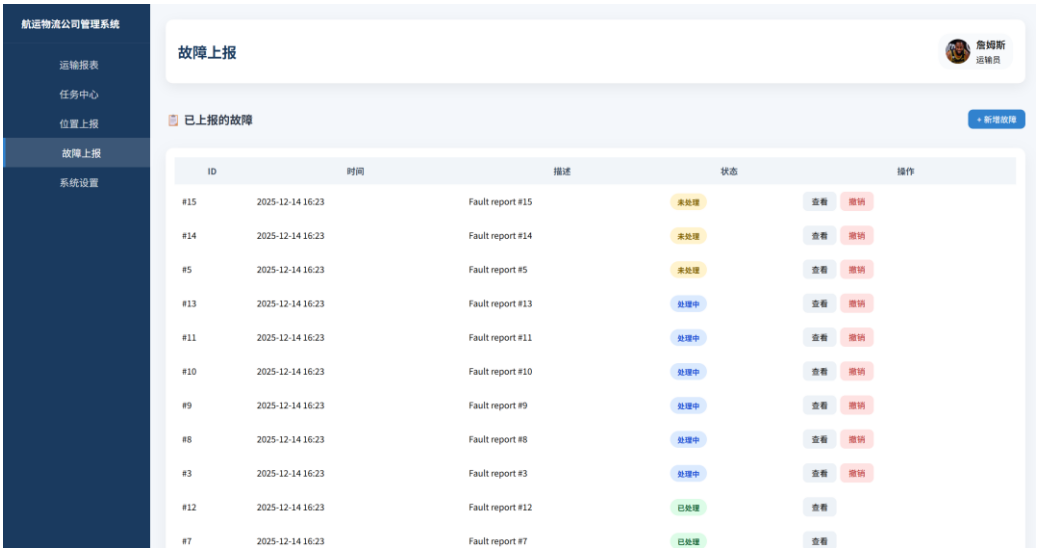


图 4.32 故障上报

对于未处理和处理中的故障，点击撤销就会删除此条故障信息，对于所有任务，都可以，查看详情。



图 4.33 故障详情

点击图 4.32 的新增故障会跳转到故障上报界面，与位置上报类似，不同之处仅在于故障上报上传的是完整的故障描述。

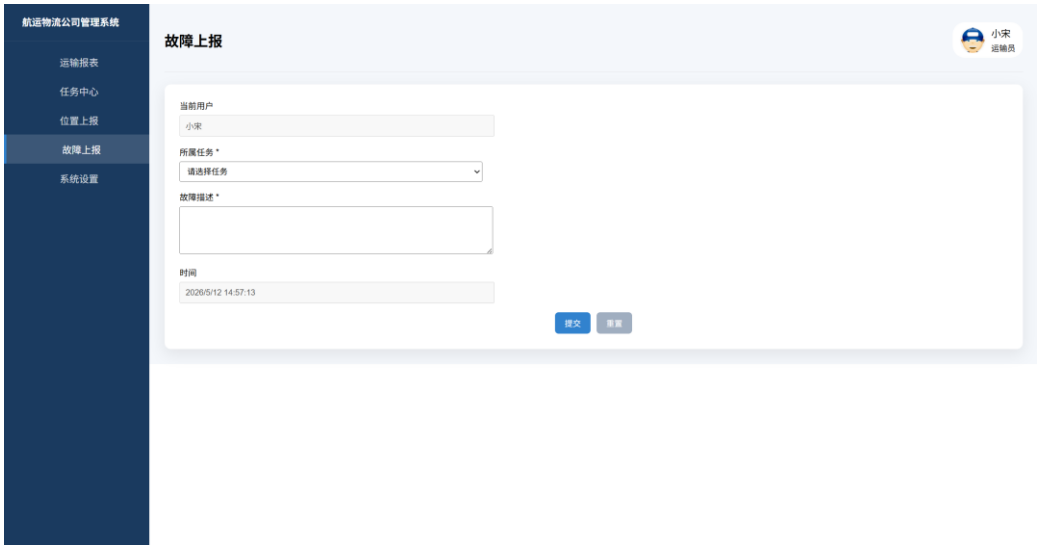


图 4.34 故障上报

4.4.5 系统设置

系统设置与操作员、指派员一致。

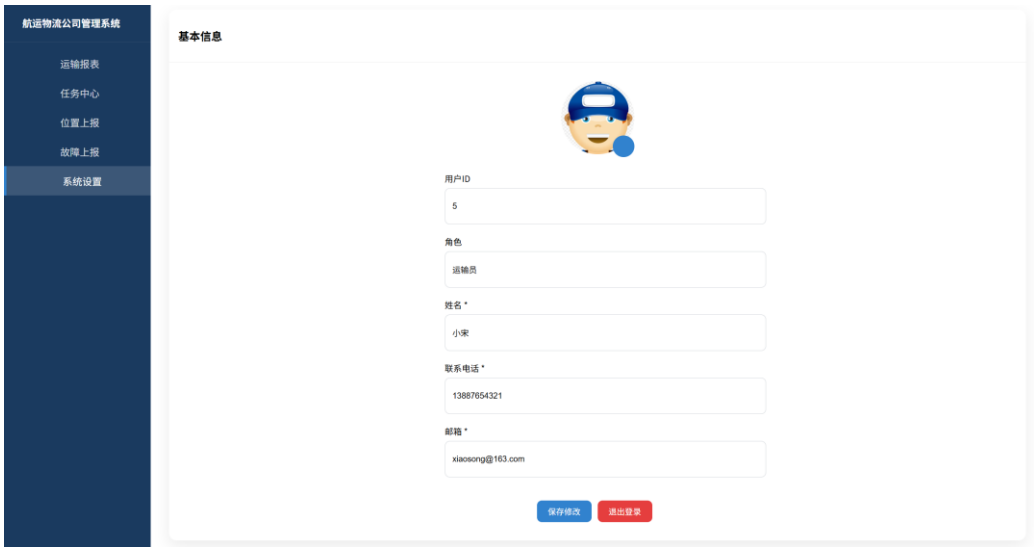


图 4.35 系统设置

4.5 管理员

4.5.1 管理员主界面

管理员主界面展示了该公司物流信息，包括订单总数，货物总价值，在途任务数以及完成率，同时支持管理员按日期筛选，可以选择固定时间段的信息。并且在界面的下方进行了数据可视化，可以查看订单随月份的趋势折线图以及订单状态分布饼图。月度订单趋势折线图设计代码：

```
const trendChart = echarts.init(trendRef.value)
trendChart.setOption({
  tooltip: {
    trigger: 'axis'
  },
  xAxis: { type: 'category', data: data.monthly_orders.map(i => i.month) },
  yAxis: { type: 'value'},
  series: [{
    data: data.monthly_orders.map(i => i.count),
    type: 'line', smooth: true, symbol: 'circle', symbolSize: 8,
    lineStyle: {
      width: 3
    },
    areaStyle: {
      opacity: 0.2
    }
  }]
})
```

订单状态分布饼图设计代码：

```
const pieChart = echarts.init(pieRef.value)
pieChart.setOption({
  tooltip: {
    trigger: 'item'
  },
  legend: {
    bottom: 0
  },
  series: [{
    type: 'pie',
    radius: ['50%', '75%'],
    itemStyle: {
      borderRadius: 8, borderColor: '#fff', borderWidth: 2
    },
    label: {
      show: true,
      formatter: '{b}: {d}%'
    },
    data: data.order_status_count.map(i => ({name: i.order_status, value: i.count}))
  }]
})
```

同时，为了保证重新加载页面后图表的正确性，引入了图表销毁机制

```
if (trendRef.value) {
    const old = echarts.getInstanceByDom(trendRef.value)
    if (old)
        old.dispose()
}
if (pieRef.value) {
    const old = echarts.getInstanceByDom(pieRef.value)
    if (old)
        old.dispose()
}
```

图 4.36 为管理员主界面

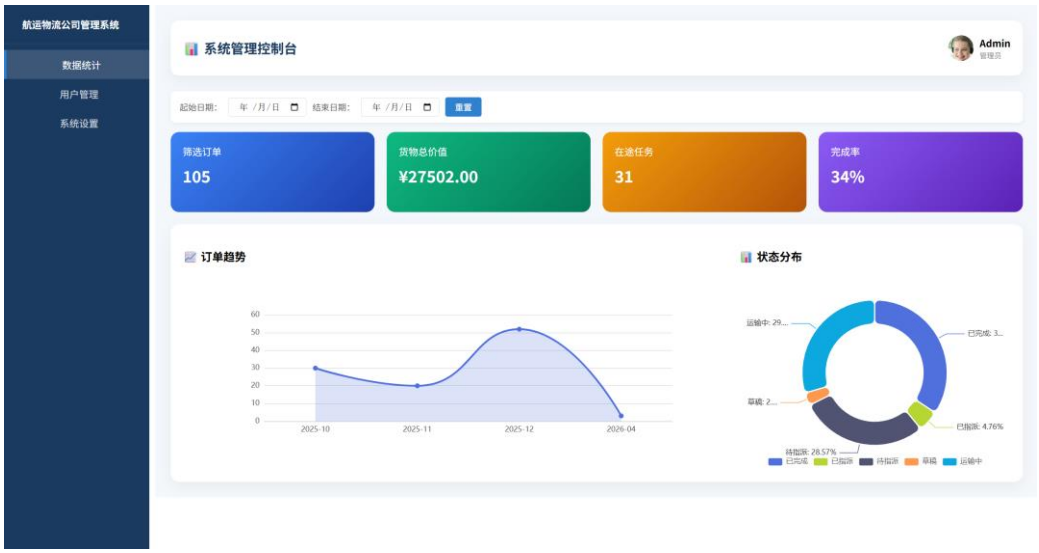


图 4.36 管理员主界面

4.5.2 用户管理

此列表下可以看到使用本系统的所有用户，展示方式与客户管理一致。

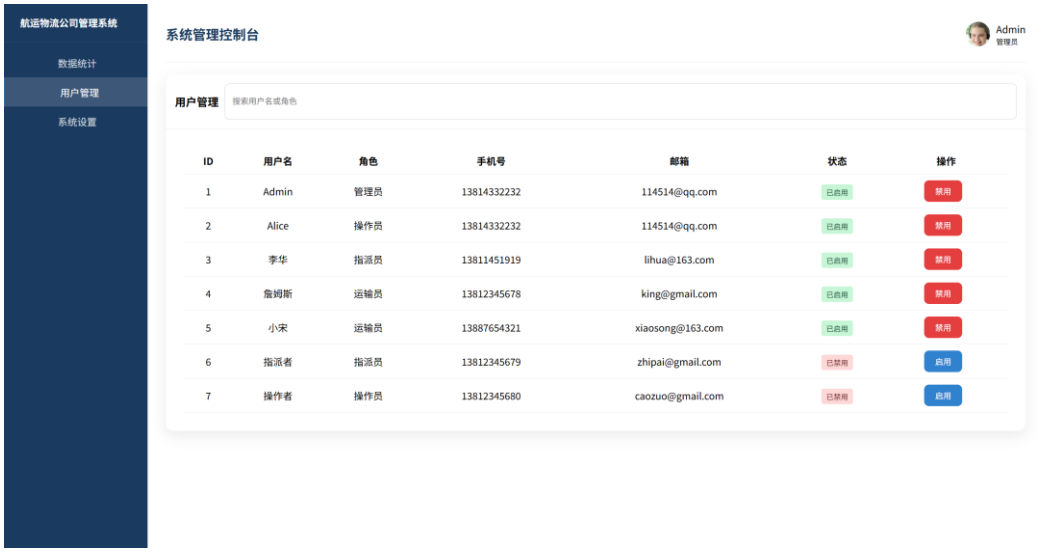


图 4.37 用户管理

点击操作栏中的“禁用”和“启用”按钮即可对应的改变用户状态。

4.5.3 系统设置

与操作员、指派员、运输员一致。

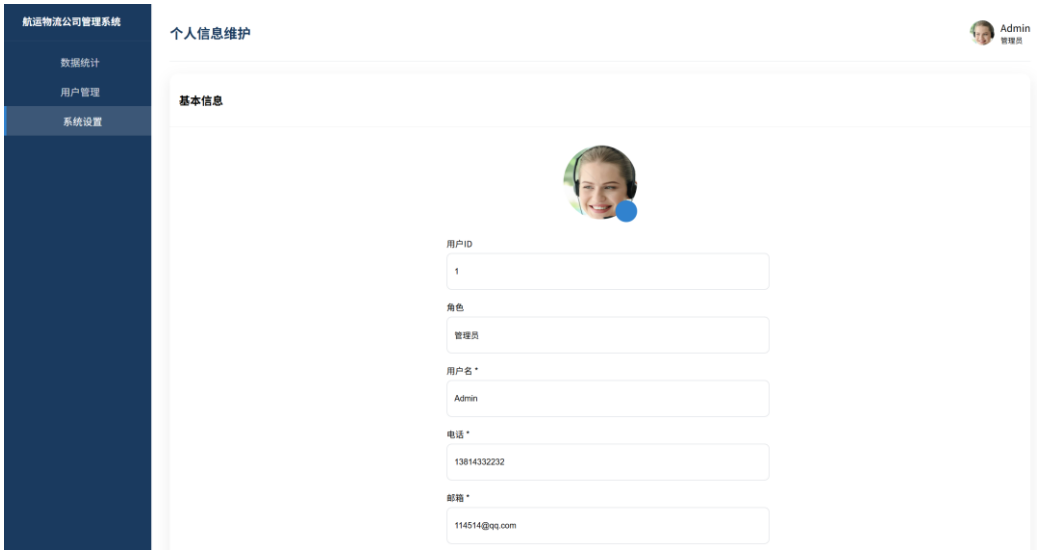


图 4.38 系统设置

5. 系统测试

5.1 测试环境

处理器 (CPU)：AMD Ryzen 9 7945HX with Radeon Graphics，2501 Mhz，16 个内核，32 个逻辑处理器。
内存 (RAM)：16 GB。
操作系统： Windows 11。
后端：Python 3.11.9 Django 5.2.12 启动：python manage.py runserver
数据库：MySql
前端：Node.js 启动：npm run dev

5.2 测试结果

5.2.1 用户登录与注册

1. 用户登录

系统用户登录功能，当输入正确的密码会提示错误，输入正确的代码进入相关界面，图 5.2 是登录失败提示

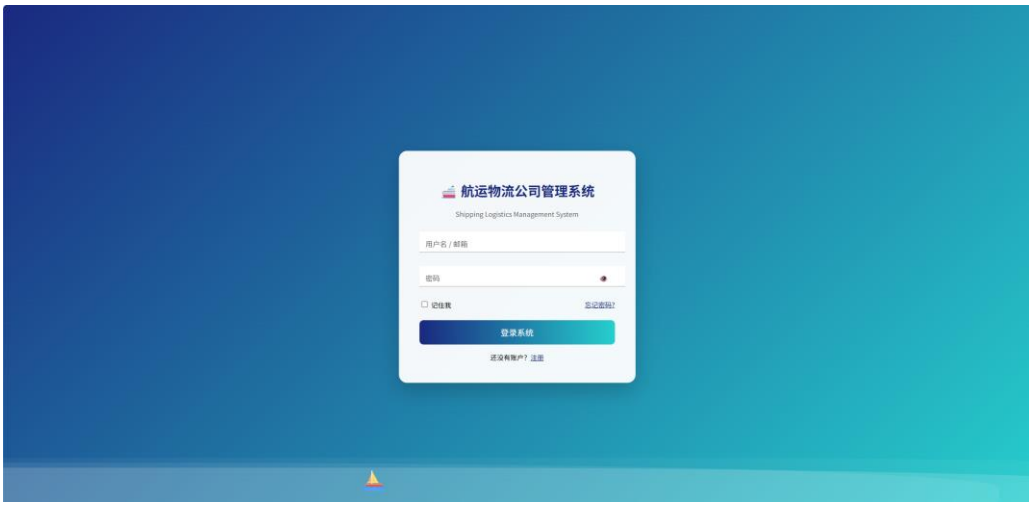


图 5.1 用户登录页面



图 5.2 密码错误提示

2. 密码找回

密码找回功能确保用户可以通过注册邮箱重置密码，两次输入不一致则弹出提示。修改成功返回登录页，修改失败如图 5.3 所示

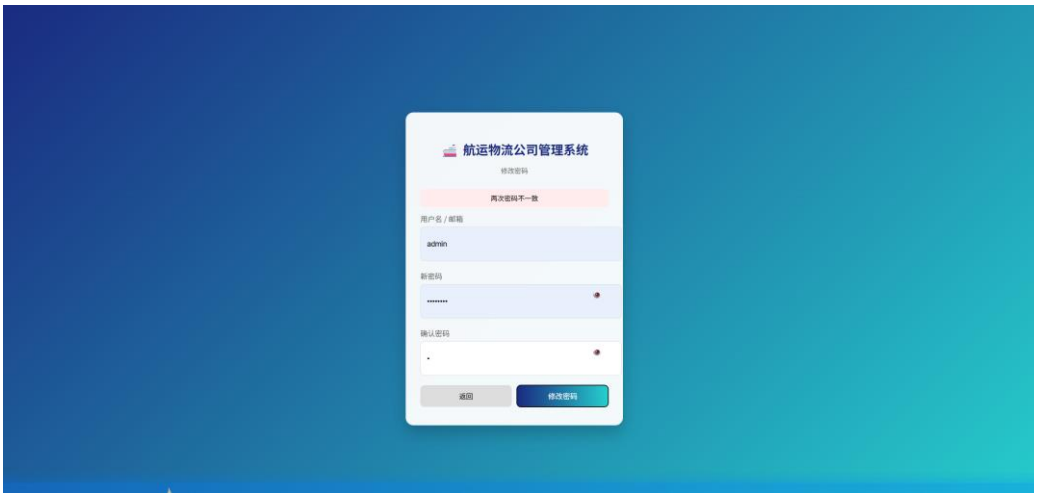


图 5.3 密码修改失败界面

3. 用户注册

用户可以进行注册，若注册信息格式不正确会拒绝，图 5.4 为注册成功，图 5.5 为

注册失败界面

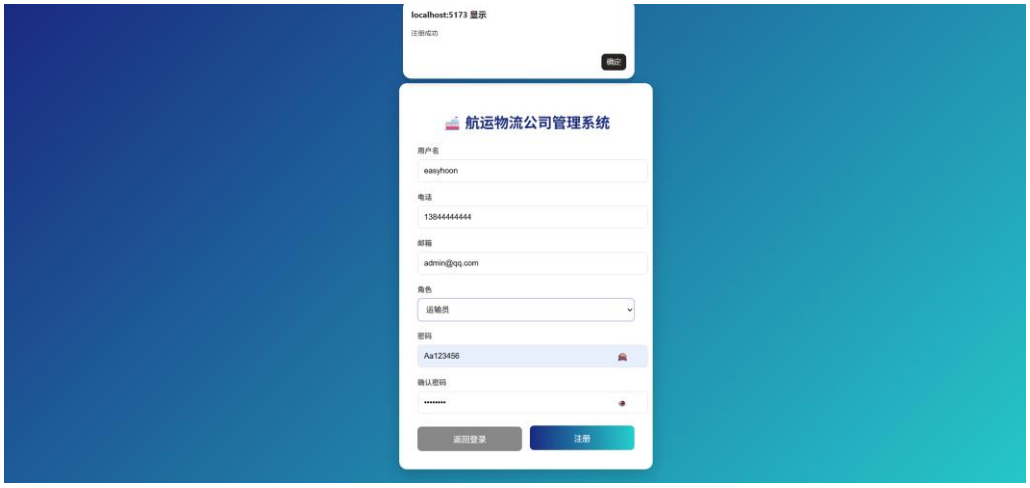


图 5.4 注册成功界面

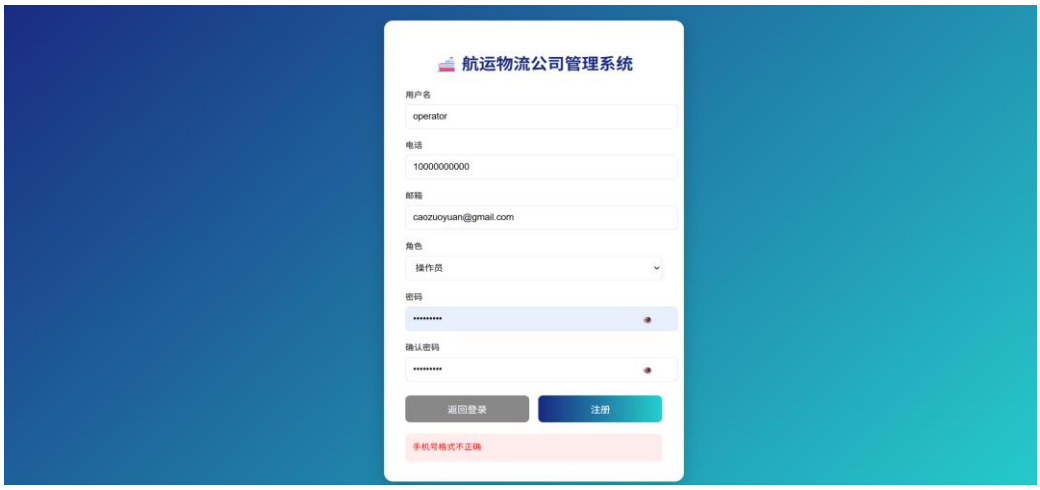


图 5.5 注册失败示例

5.2.2 管理员功能

1. 管理员主界面

所有时间相关的报表都可通过时间选择确定报表内容的起始时间，图 5.6 和 5.7 为时间修改前后展示

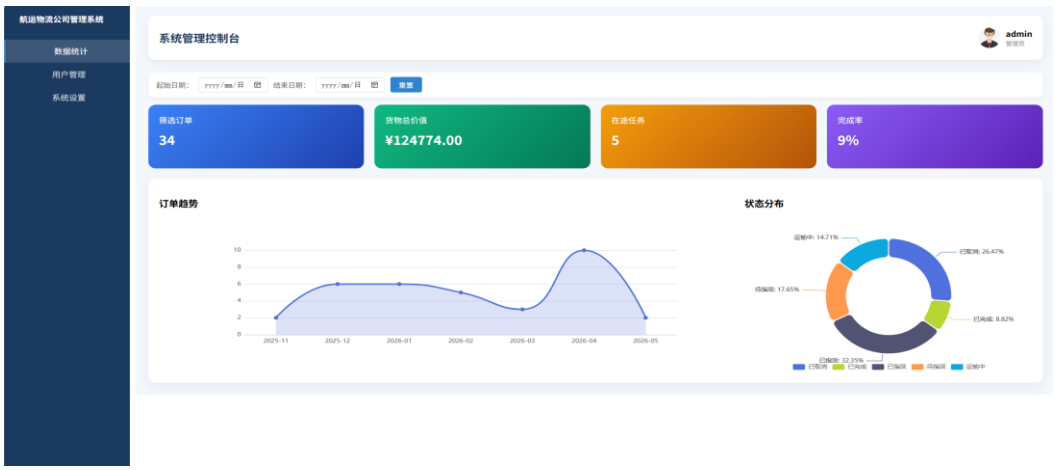


图 5.6 管理员主界面报表展示

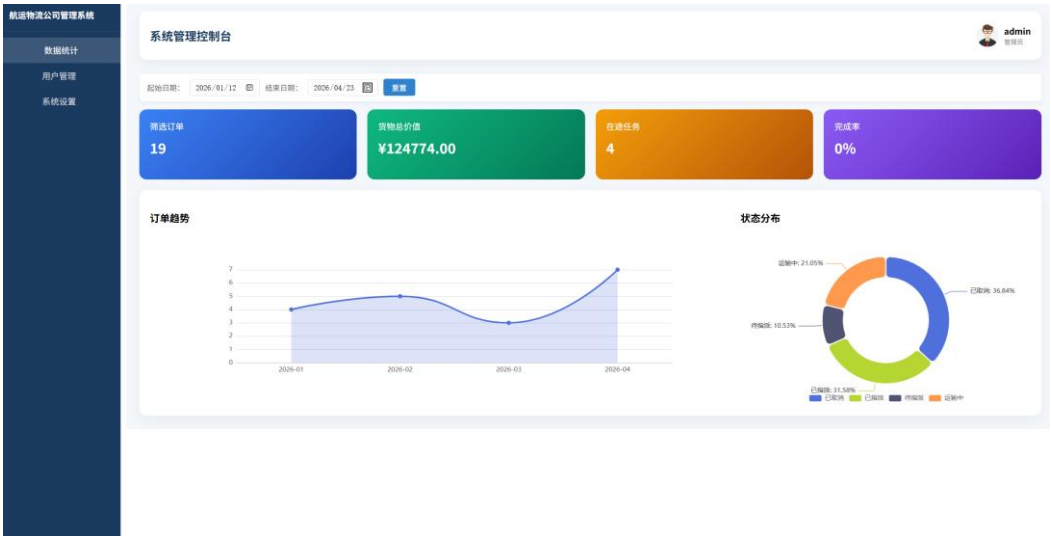


图 5.7 修改日期范围更新报表

2. 用户管理

验证管理员对系统用户账号的管理功能，包括禁用和启用操作。图 5.9 为被禁用账号无法登录

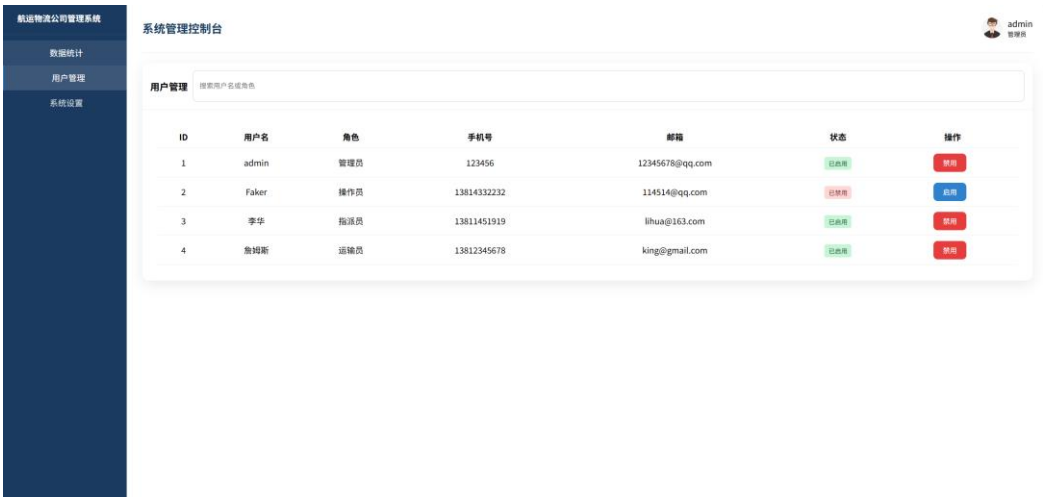


图 5.8 用户管理-禁用账号

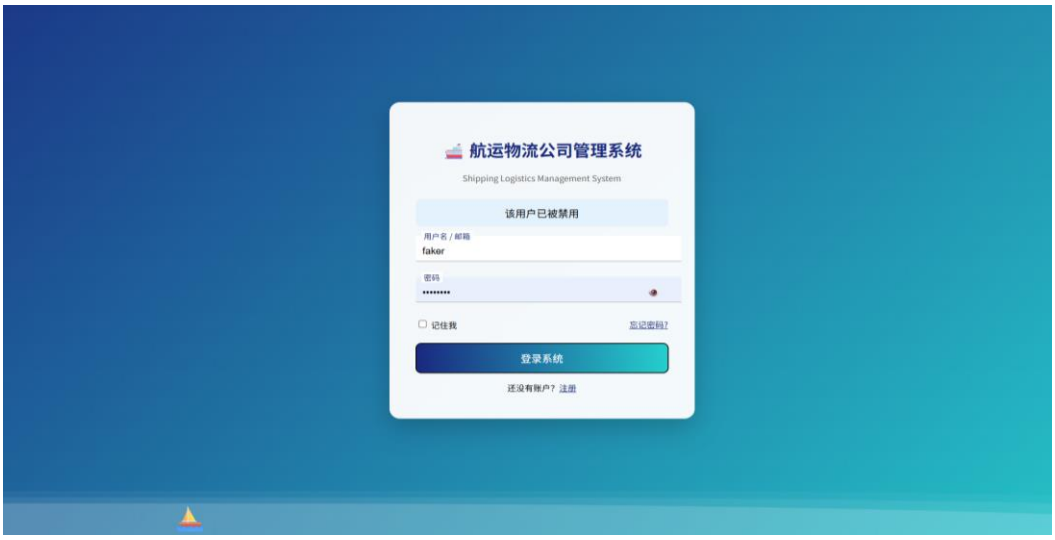


图 5.9 被禁用账号无法登录

3. 个人信息维护

所有账户都可进行个人信息的维护， 图 5.10 为修改成功

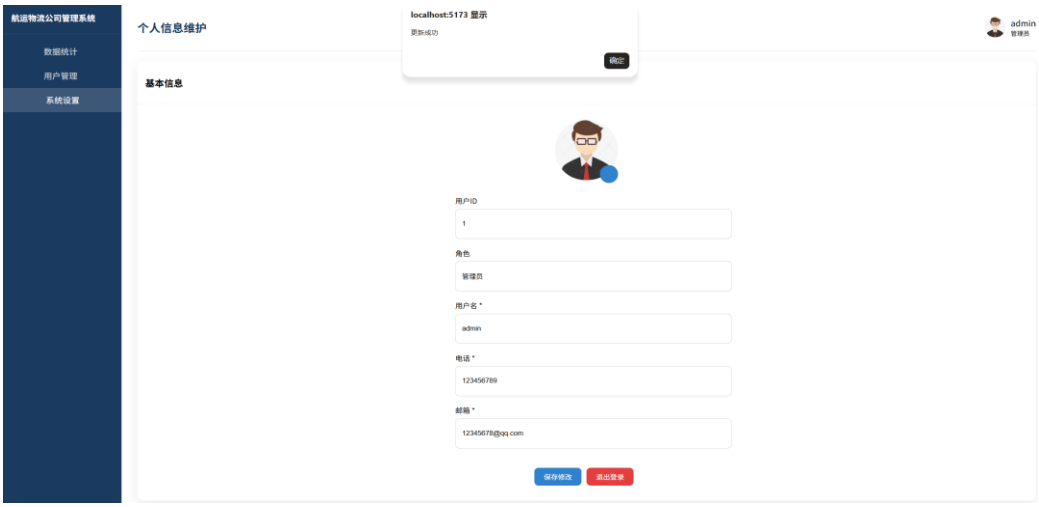


图 5. 10 保存修改成功提示

5. 2. 3 操作员功能

1.客户管理

进入页面后可通过搜索框模糊搜索客户，并可以创建、编辑、管理客户。图 5.11 可见用户已被修改

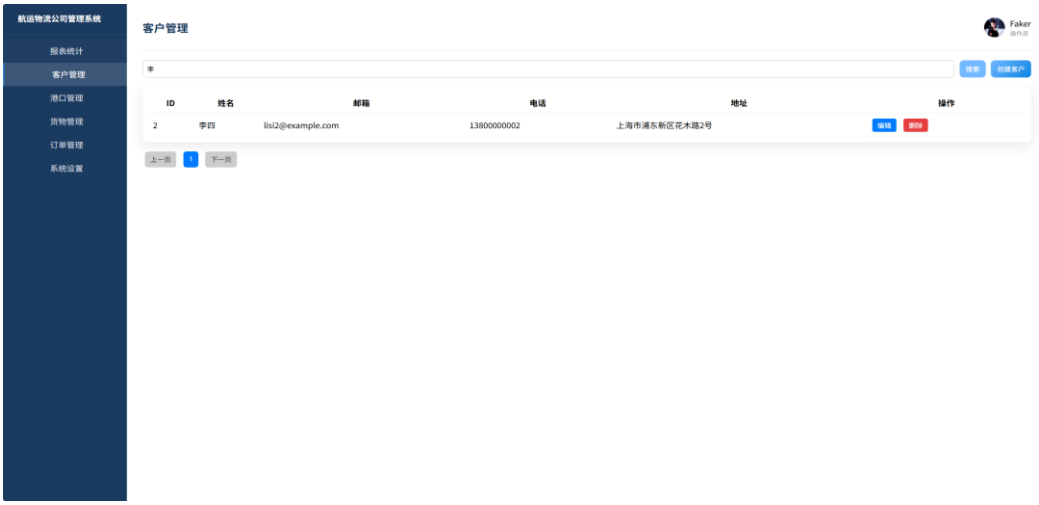


图 5.11 客户查询界面

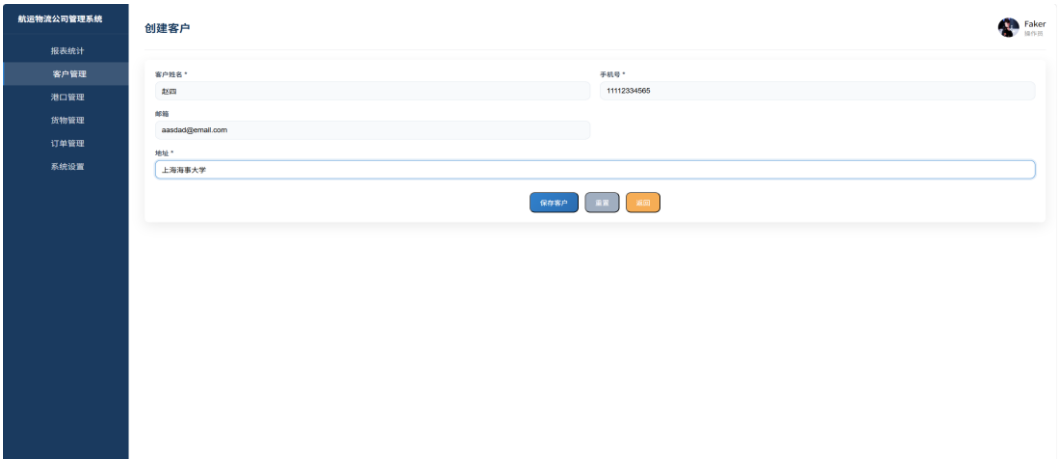


图 5.12 客户创建界面

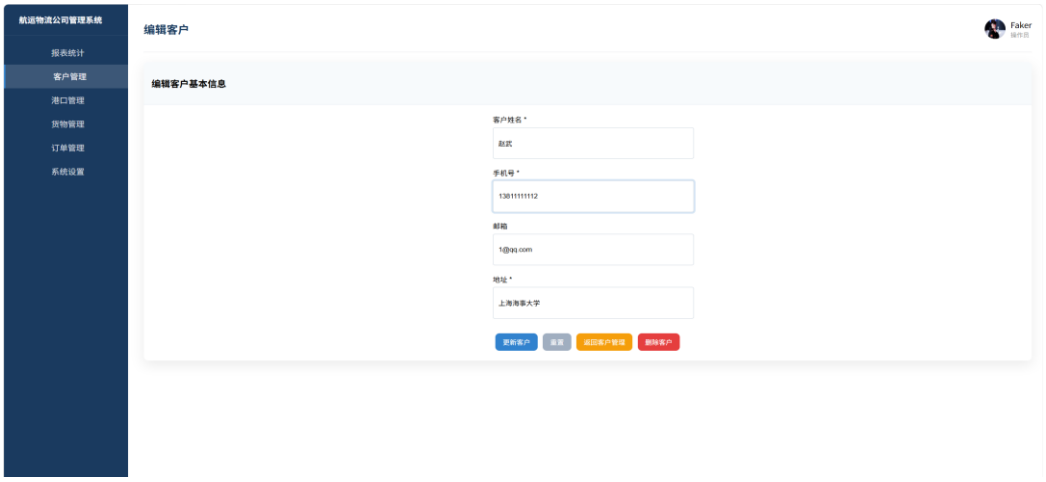


图 5.13 客户编辑界面

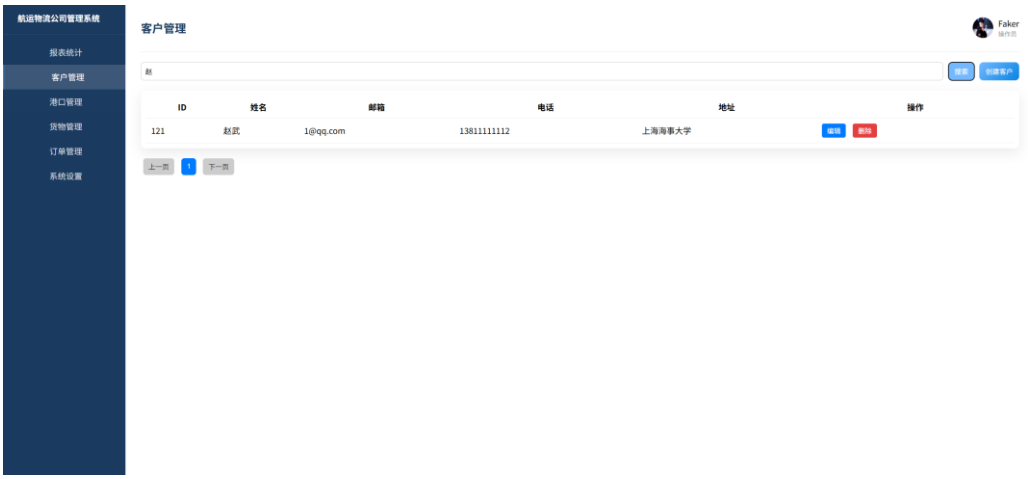


图 5.14 客户更新后界面

港口、货物、订单增删改查同上。

5.2.4 指派员功能

1.任务管理功能

在任务管理界面，指派员可以创建具体的任务，并对其进行管理，任务分为很多状态，运输员在完成任务后状态将更新。具体更新将在后文运输员功能进行展示。

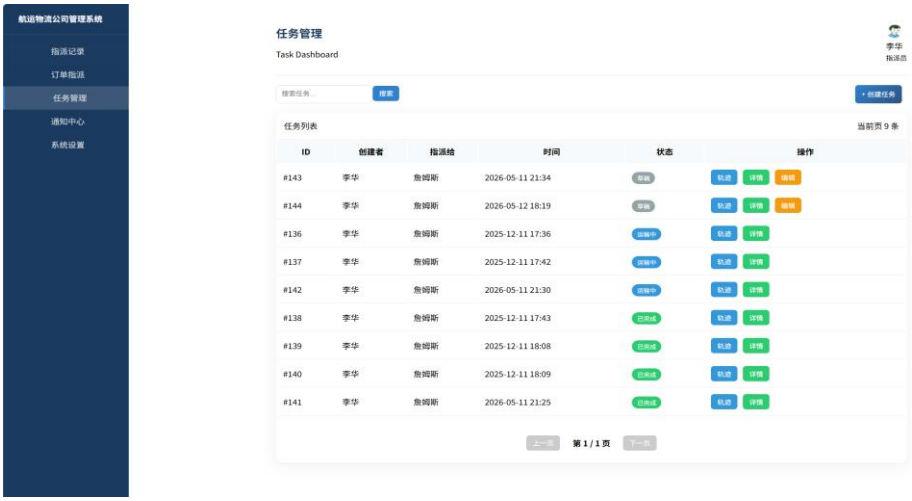


图 5.15 任务管理界面

指派员可以点击轨迹，从而查看运输员上报的该任务轨迹。如图 5.16 所示

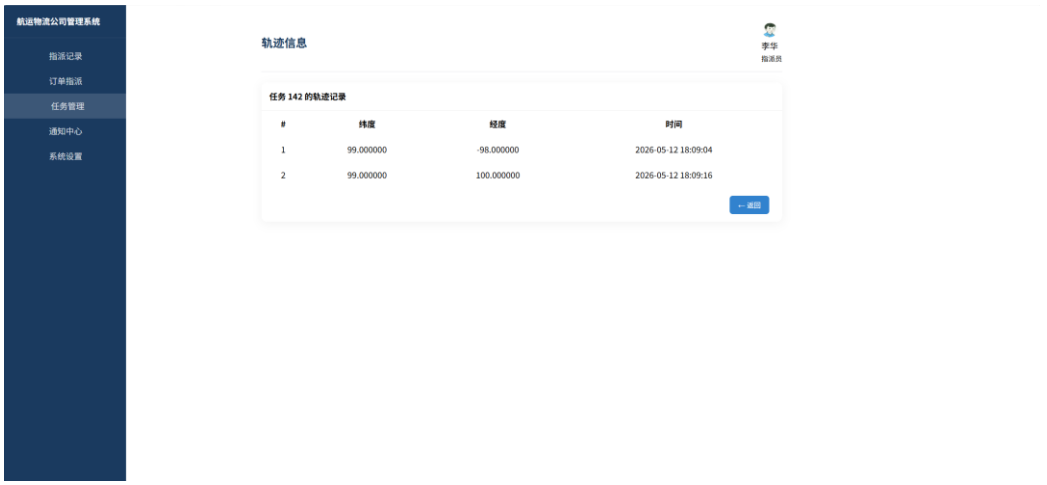


图 5.16 轨迹信息界面

2.订单指派功能

该界面对应了具体的客户的订单，指派员可以把客户的订单指派给草稿状态的任务。执行中的任务不可以被指派。可见图 5.17 中只出现草稿状态任务。

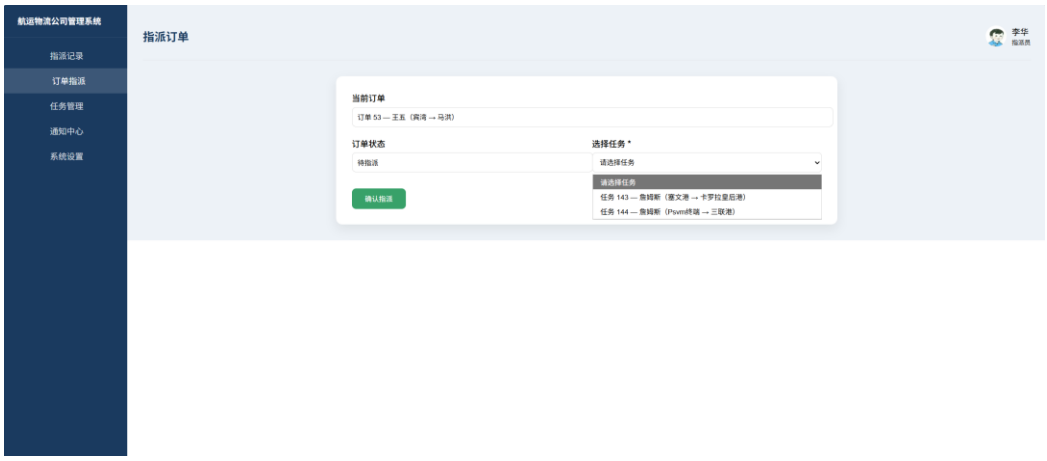


图 5.17 订单指派界面

3.通知中心

通知中心展示下层运输员报告的故障，指派员可以点击故障详情，处理运输员上报的故障。图 5.19 为点击处理后状态



图 5.18 消息中心界面



图 5.19 处理后界面

5.2.5 运输员功能

1.位置上报功能

运输员可上报位置，图 5.20 为上报成功，5.21 为指派员端记录更改

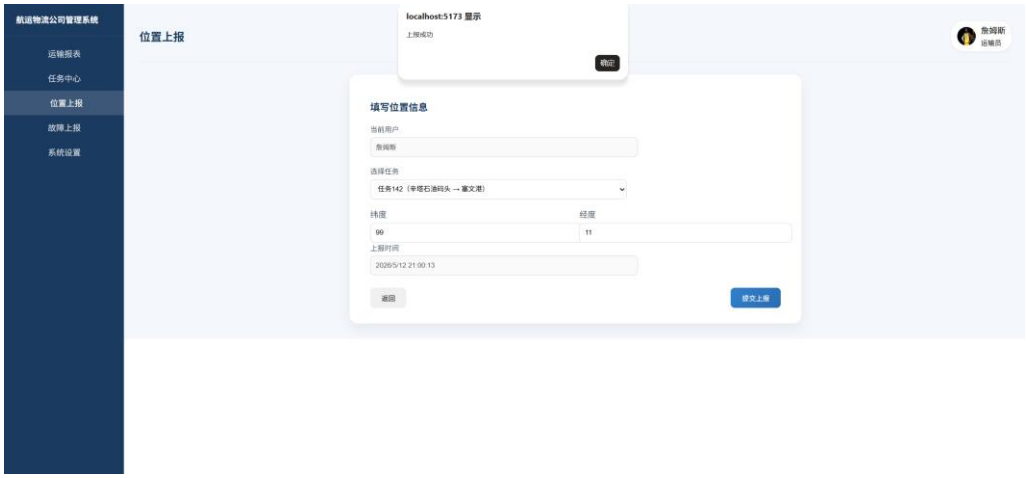


图 5.20 位置上报成功



图 5.21 指派员端新增轨迹

2.故障上报功能

运输员可上报故障，让指派员来处理。图 5.22 为上报成功，5.23 为指派员端增加故障

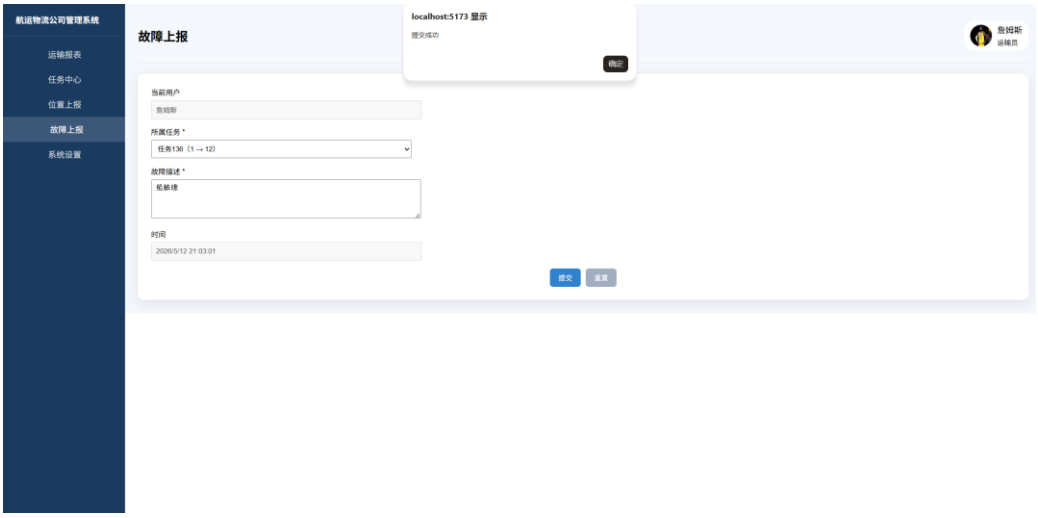


图 5.22 故障上报成功

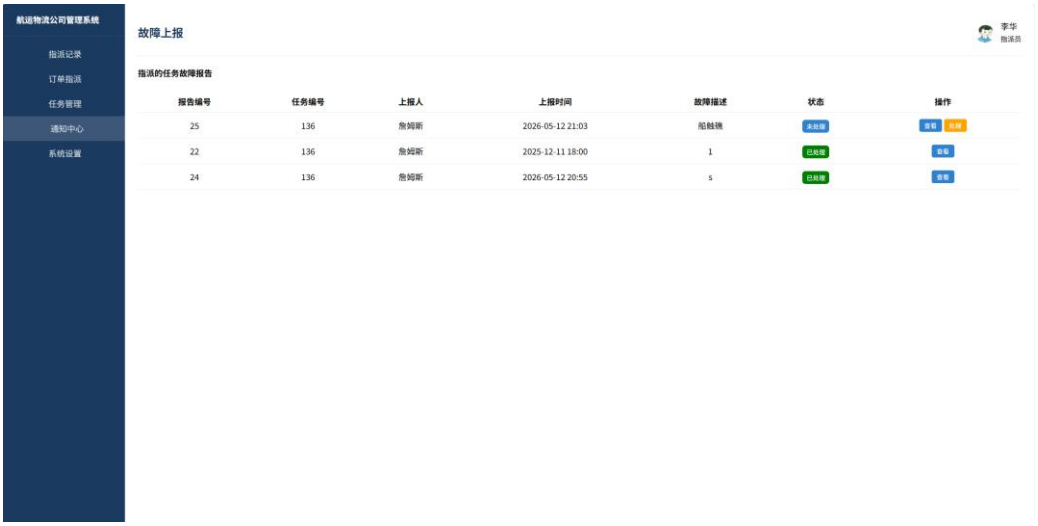


图 5.23 指派员端新增故障

6. 系统改进

6.1 课程设计存在的问题及解决思路

在本次《航运物流公司管理系统》的开发过程中，虽然系统基本完成了订单管理、任务指派、运输跟踪、故障上报以及数据可视化等核心功能，但在实际开发过程中仍然遇到了许多问题。首先，由于系统采用前后端分离架构，前端 Vue 与后端 Django 在接口联调阶段经常出现数据格式不一致、接口路径错误以及跨域请求等问题，导致部分功能无法正常运行。针对这些问题，小组通过统一接口返回格式、规范 API 命名规则，并使用 Axios 统一封装请求模块，同时结合 Django 的跨域配置，逐步解决了前后端通信问题。其次，在数据库设计阶段，由于业务实体较多，订单、任务、货物以及轨迹之间关系复杂，初期设计中存在数据冗余和关联不清晰的问题。后续通过重新梳理 E-R 图与关系模型，增加中间表并规范主外键关系，使数据库结构更加合

理。此外，在实现数据可视化时，由于图表在窗口缩放或页面切换时容易出现渲染异常，小组通过引入 ECharts 的 resize 与 dispose 机制，保证了图表展示的稳定性。最后，在用户体验方面，系统初期表单校验不足，容易出现错误数据提交的问题，因此后续增加了前后端双重数据校验、错误提示以及操作确认机制，从而提高了系统的稳定性与易用性。

6.2 进一步改进方向

6.2.1 增加地图以进行更丰富的可视化分析

目前已存在轨迹上报功能，但是仅为数据展示，后续希望能引入地图机制，实时展示任务状态，增强管理效率。

6.2.2 完善通知系统（如短信、邮件提醒）

系统现阶段仅提供站内消息。后续可扩展：订单指派邮件提醒运输员、任务完成自动通知客户、故障紧急提醒提高系统的实时响应能力。

6.2.3 提升前端用户体验

未来可以添加：列表支持更多筛选与导出格式、表单自动补全、错误提示更智能、订单与任务支持拖拽指派。让界面更加贴近真实航运企业使用习惯。

6.2.4 加入异常自动处理与风险预警

结合运输人员的轨迹与历史异常数据，可以预测风险，例如：运输偏离路线自动预警、长时间无定位上报提醒、故障多发运输员自动提示管理层，提升系统智能化水平。

7. 小结

本次《航运物流公司管理系统》的课程设计围绕航运物流业务的实际需求，完成了从系统需求分析、总体架构设计、数据库建模到前后端功能实现与系统测试的完整开发流程。系统采用 Vue 3 与 Django 前后端分离架构，结合 MySQL 数据库与 ECharts 数据可视化技术，实现了订单管理、任务指派、运输跟踪、故障处理及统计分析等核心业务模块，基本满足了航运物流企业数字化管理的需求。

通过本项目的实践，不仅加深了对软件工程开发流程的理解，也进一步掌握了前后端分离开发模式下接口设计与数据交互的实现方法。同时，在系统开发与调试过程中，对权限控制、数据库关系设计以及前端可视化展示等关键技术有了更为深入的认识。

总体来看，本系统在功能完整性与结构设计方面基本达到了课程设计要求，但在系统智能化、地图可视化以及消息实时推送等方面仍有进一步优化空间。未来可在现有基础上持续扩展功能模块，提升系统的实用性与工程化水平。

参考文献

[1] Tongzon J L, Nguyen H O. Effects of port-shipping logistics integration on technical and allocative efficiency[J]. The Asian Journal of Shipping and Logistics, 2021, 37(2): 109-116.

[2] Muhanguzi E. Design and implementation of a web based flight and shipping logistics management system: a case study of Kenlloyd Logistics[J]. 2013.

[3] Vue.js 官方文档. Vue 3 文档[EB/OL]. https://cn.vuejs.org/,

[4] Django 文档 https://docs.djangoproject.com/zh-hans/6.0/

[5] 夏帮贵 Python Web 开发基础教程（Django 版）（微课版）人民邮电出版社，2020

[6] 王珊, 杜小勇，陈红. 数据库系统概论(第 6 版)[M]. 北京: 高等教育出版社, 2022.

[7] MySQL Documentation. MySQL 8.0 Reference Manual[EB/OL]. 2024. https://dev.mysql.com/doc/refman/8.0/en/

[8] 林自葵 刘建生物流信息管理 第 2 版 机械工业出版社，2021

[9] 唐仲华,嘉特航运物流信息平台系统 V1.0.广西壮族自治区,贵港市嘉特电子科技有限公司,2019-02-20.

[10] 管治武.基于嵌入式技术的船舶航运物流系统构建探讨[J].船舶物资与市场,2021,29(09):1-2.DOI:10.19727/j.cnki.cbwzysc.2021.09.001.

[11] 谷小强.物流管理在工程项目中的应用策略探讨[J].中国航务周刊,2025,(47):60-62.

致谢

在本次《航运物流公司管理系统》课程设计的完成过程中，我得到了许多老师和同学的帮助与支持，在此谨向他们致以诚挚的感谢。

首先，衷心感谢指导教师金世双老师在整个课程设计过程中给予的悉心指导与严格要求。从选题确定、系统架构设计到论文撰写与修改完善，老师都给予了细致的指导和宝贵的建议，使我能够不断优化系统设计思路，规范论文结构，提高项目实现质量。老师严谨的治学态度和认真负责的工作精神让我受益匪浅。

其次，感谢信息工程学院的各位任课老师。在大学阶段的学习中，各位老师系统地讲授了数据库原理等相关课程，为本次课程设计的顺利完成奠定了坚实的理论基础。

同时，也要感谢本次课程设计小组成员冯方舟同学，在项目开发过程中与我密切配合，承担了前端界面设计、报表展示以及系统测试等工作。正是通过良好的团队协作，我们才能高效完成系统的整体开发与调试。

最后，感谢在项目开发过程中给予我帮助的同学与朋友，在遇到技术难题时给予的建议和支持，使我能够顺利解决开发过程中遇到的各种问题。

本次课程设计不仅是对所学知识的一次综合运用，也是一次宝贵的实践经历，使我在技术能力与工程实践方面都有了明显提升。再次向所有给予帮助的人表示衷心的感谢。